

KAMP 제조AI데이터셋 활용 권한

No. GB20260923-PU1789290667-375470



⚠ 제조AI데이터셋 활용 주의사항

연구/공식적으로 사용하실 때에는 꼭 아래와 같이 'KAMP 출처(reference)'를 남겨주시고, 인용 시 활용한 내용과 문서 등은 아래 이메일로 보내주시기를 바랍니다.
(E-mail : kamp@kaist.ac.kr)

국문 출처 표기 양식

중소벤처기업부, Korea AI Manufacturing Platform(KAMP), 소성가공 예지보전 AI 데이터셋, 스마트 제조혁신추진단(주)인터엑스, 2022.12.23., www.kamp-ai.kr

영문 출처 표기 양식

Ministry of SMEs and Startups., and KAIST (Korea Advanced Institute of Science and Technology). (2022, December 23). Plastic forming predictive maintenance AI dataset. Korea AI Manufacturing Platform(KAMP). <https://www.kamp-ai.kr/>

제조AI데이터셋 개요

제조AI데이터셋명 : 소성가공 예지보전 AI 데이터셋

업종 : 뿌리(소성가공) | **목적** : 예지보전 | **유형** : csv

알고리즘 : LSTM-Autoencoder

사용조건 : 콘텐츠 변경허용

제공기관 : 스마트제조혁신추진단 (수행기관 : (주)인터엑스)

등록일 : 2022-12-23

이용자 정보

회원 ID : MSS_jdmin2003

다운로드일 : 2026-09-23 14:51:04

활용목적 : 제조AI 교육 및 강의

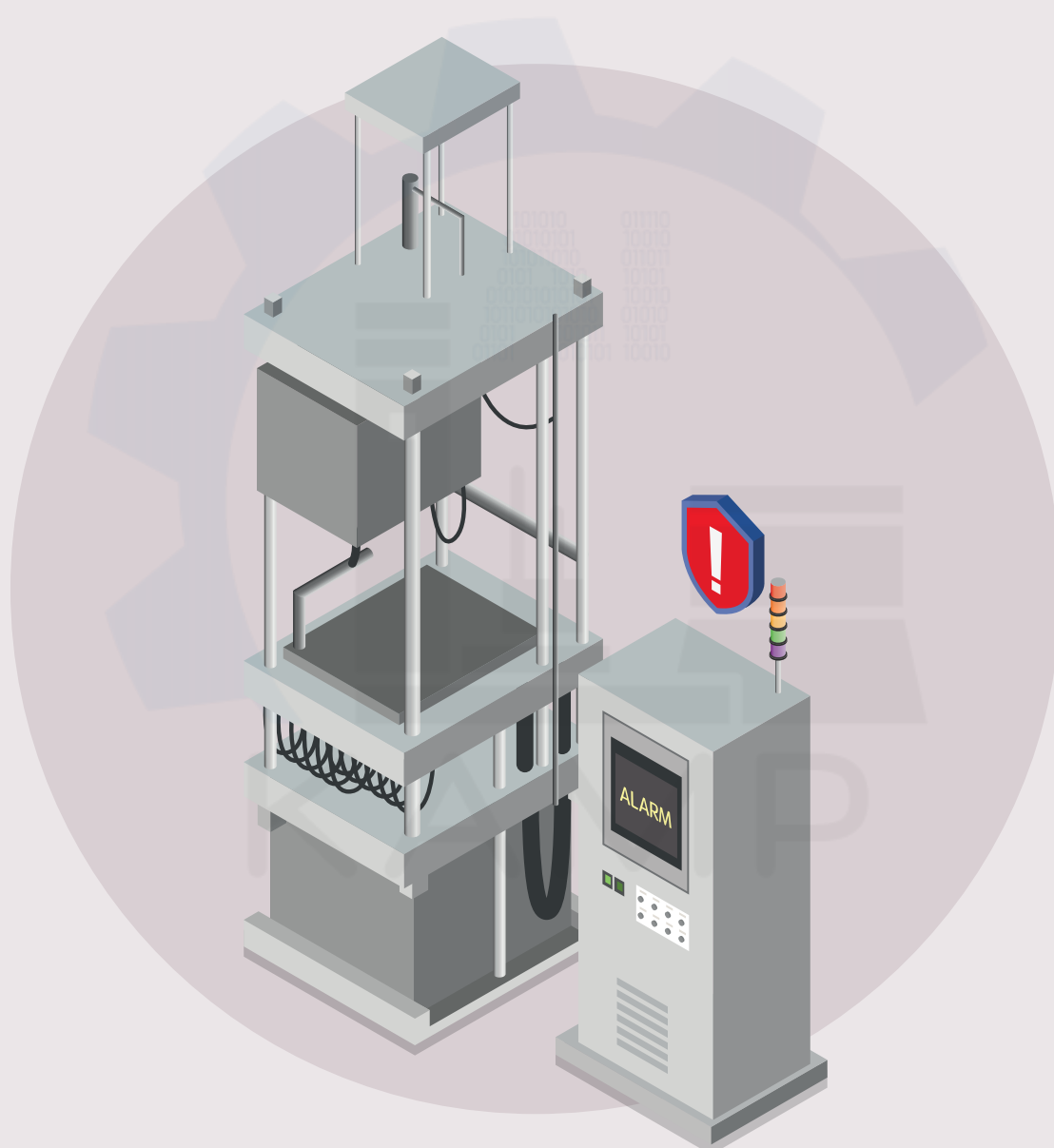
인공지능 제조 플랫폼(KAMP, Korea AI Manufacturing Platform)에서 다운로드한 제조AI데이터셋의 저작권을 위하여 문서 고유 번호와 워터마크를 부여하여, 해당 문서의 표준을 준수하였음을 증명합니다.

운영기관 : KAIST 제조AI빅데이터센터

주소 : 대전광역시 유성구 문지로 193 KAIST 문지캠퍼스 행정동

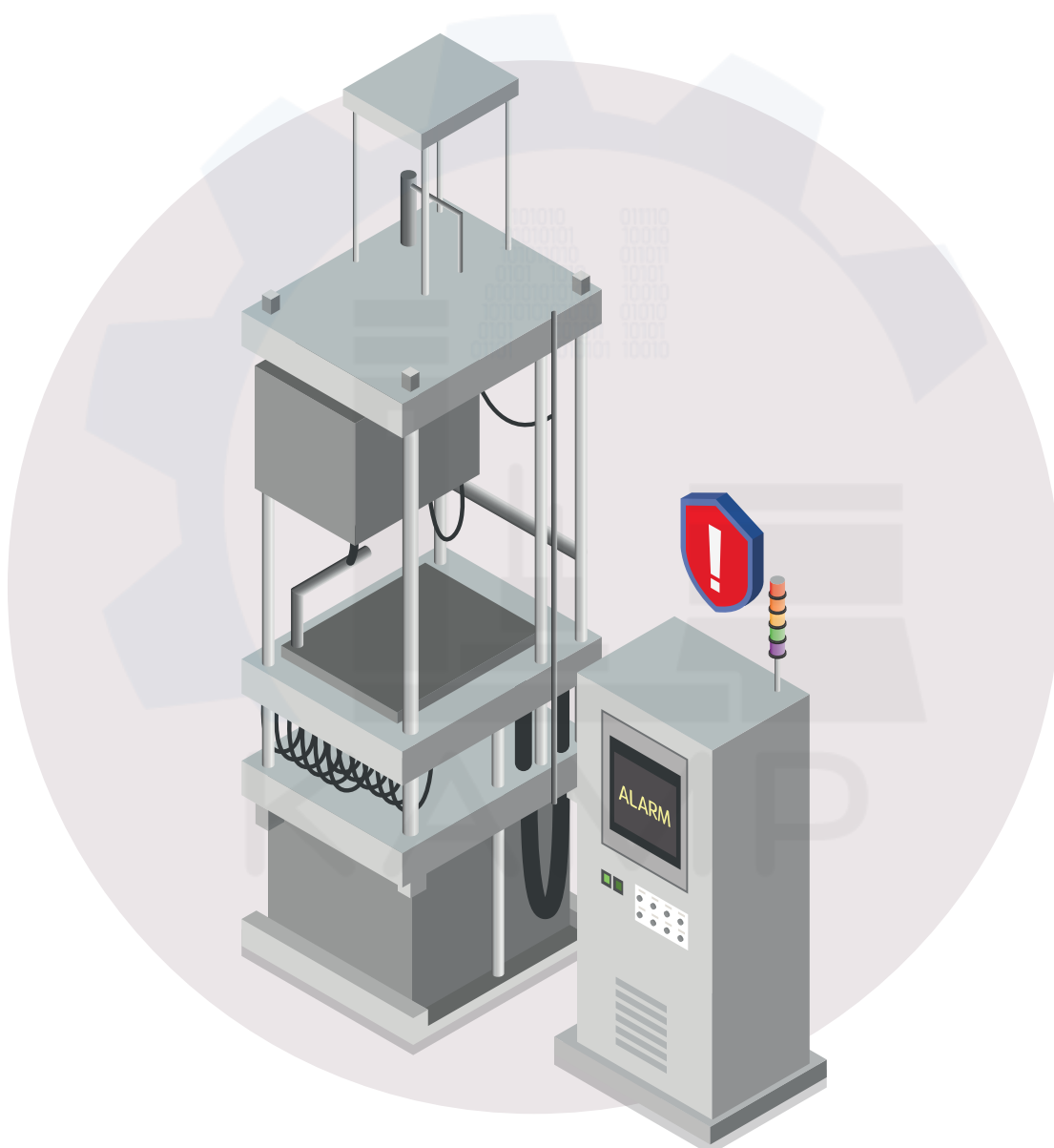
전화번호 : 042-350-1331 | 이메일 : kamp@kaist.ac.kr

「소성가공 예지보전 AI 데이터셋」 분석실습 가이드북



「소성가공 예지보전 AI 데이터셋」

분석실습 가이드북



Contents

1 분석요약	04
2 분석 실습	
1. 분석 개요	05
1.1 분석 배경	05
1) 공정(설비) 개요	
2) 이슈사항(Pain point)	
1.2 분석 목표	08
1) 분석 목표	
2) 데이터 정의 및 소개	
3) 데이터 분석 기대효과	
4) 시사점(implication) 요약기술	
2. 분석실습	10
2.1 제조데이터 소개	10
1) 데이터 수집 방법	
2) 데이터 유형/구조	
3) 데이터 (품질) 전처리	
2.2 분석 모델 소개	13
1) 데이터 흐름 및 인공지능 모델 적용 흐름도	
2) AI 분석모델	
2.3 분석 체험	18
1) 필요 SW, 패키지 설치 방법 및 절차 가이드	
2) 분석 단계별 프로세스 - Flow Chart	
[단계 ①] 데이터 준비	
[단계 ②] 데이터 탐색	
[단계 ③] 데이터 정제	
[단계 ④] 알고리즘 선택	
[단계 ⑤] 학습, 평가 데이터 준비	
[단계 ⑥] 모델 정의 및 생성	
[단계 ⑦] 모델 학습 및 평가	
3. 유사 타 현장의 「소성가공 예지보전 AI 데이터셋」 분석 적용	41
부 록	
분석환경 구축을 위한 설치 가이드	43
데이터 품질 전처리 [실습 코드]	56



「소성가공 예지보전 AI 데이터셋」 분석실습 가이드북

- 필요 SW : Python, Anaconda, Jupyter Notebook
- 필요 패키지 : pandas, sklearn, matplotlib, numpy, seaborn, tensorflow
- 분석 환경 : [CPU] AMD Ryzen 9 5950x, [Memory] 128GB, [GPU] Nvidia RTX 3080 * 2
- 필요 데이터 : press_data_normal.csv, outlier_data.csv

1 분석요약

No	구 분		내 용
1	분석 목적 (현장 이슈, 목적)		소성가공 공정에서 발생하는 유압펌프 고장을 조기에 탐지하기 위해 AI를 사용하여 분석한다. 유압 유압펌프에서 발생하는 진동 및 전류 데이터의 상관관계를 파악하고 시계열 딥러닝 알고리즘을 사용하여 유압펌프 이상 데이터를 탐지한다.
2	데이터셋 형태 및 수집방법		1) 분석에 사용된 변수 : AI0_Vibration, AI1_Vibration, AI2_Current, Equipment_state 2) 데이터 수집 방법 : PLC 및 센서를 통해 실시간 데이터를 수신하여 수집된 시간을 기준으로 CSV 파일 형태로 저장한다. 3) 데이터셋 파일 확장자 : CSV
3	데이터 개수 데이터셋 총량		- 데이터 개수 : 정상데이터 100,000개(column수 5개 * row수 20,000개), 이상데이터 3,000개(column수 5개 * row수 600개) - 데이터셋 총량 : 정상데이터 1.15MB, 이상데이터 37.9KB
4	분석적용 알고리즘	알고리즘	LSTM-Autoencoder
		알고리즘 간략소개	LSTM-Autoencoder는 대표적인 시계열 딥러닝 알고리즘인 LSTM과 이상치탐지에 주로 사용되는 Autoencoder를 결합시킨 알고리즘으로서 정상 시계열 데이터로만 학습을 진행한다. 이후 학습된 모델에 이상 시계열 데이터가 입력되었을 때 발생하는 재구성 오차를 바탕으로 이상치를 탐지하는 알고리즘이다.
5	분석결과 및 시사점		- 소성가공 공정에서 수집된 설비 데이터를 LSTM-Autoencoder를 사용하여 이상 작동 탐지 모델을 개발하였다. - 본 분석을 통해 소성가공 공정의 유압유압펌프 고장을 조기에 탐지할 수 있다. 이러한 예지보전을 통해 설비 고장으로 인한 제품 불량 발생 및 공정 가동 중단을 예방할 수 있다.



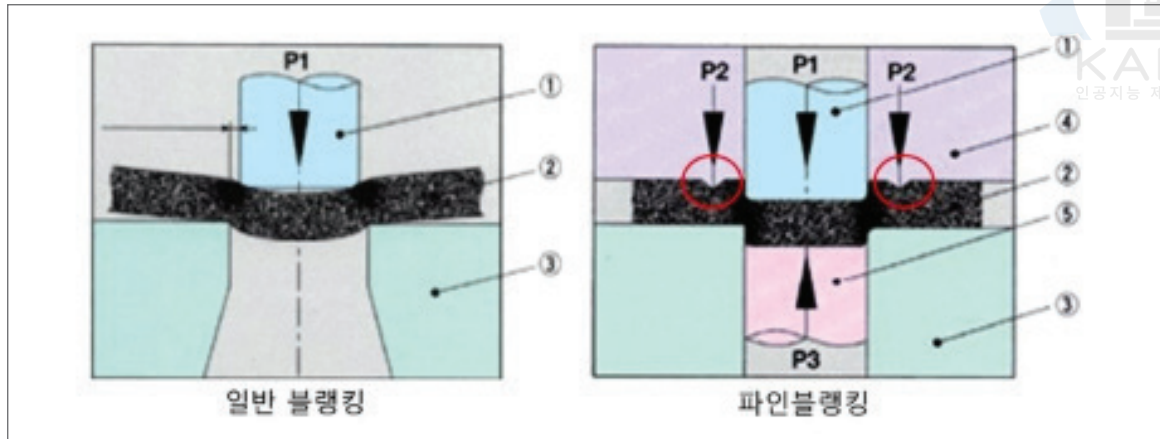
2 분석 실습

1. 분석 개요

1.1 분석 배경

1) 공정(설비) 개요

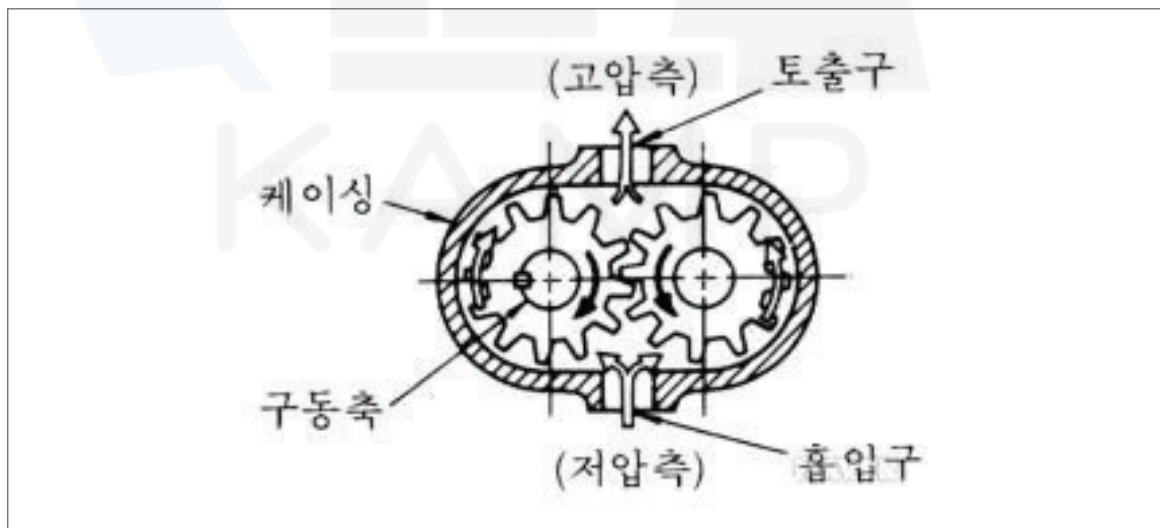
- 소성가공은 고체를 일정한 힘으로 변형시키면 원래의 형태로 돌아가지 않는 성질(소성)을 이용한 가공 방법으로 변형을 가하는 온도에 따라 냉간가공과 열간가공으로 나뉜다. 열간가공은 금속을 가열하여 연화시켜 가공하는 방법으로, 적은 힘으로도 금속을 변형시킬 수 있고 같은 힘으로 한번에 큰 변형을 줄 수 있다. 즉, 적은 공정으로 제품을 만들 수 있어 경제적이거나, 가공 표면이 불균일하고 치수정밀도 또한 좋지 않아 잘 사용되지 않고 있다. 반면에, 냉간가공은 상온에서 실시되며 변형을 위해 많은 힘이 필요하지만 정확한 치수정밀도를 얻을 수 있고 표면 또한 매끄럽게 가공되는 장점이 있다.
- 소성가공은 방법에 따라 분류할 수 있다. 재료를 가열하고 두들기는 단조, 재료를 거꾸집 속에 밀어 넣고, 거꾸집에 뚫린 구멍으로 밀어내는 압출, 회전하는 2개의 롤 사이로 재료를 통과시켜 판재를 만드는 압연, 틀에 재료를 통과시켜 잡아당겨서 단면의 감소를 꺾이는 신선, 재료에 형을 대고 눌러 원하는 형상을 만드는 프레스 등이 있다. 본 분석에서는 프레스 가공공정의 데이터를 사용한다.
- 프레스 가공은 냉간 소성가공의 한 종류로서 가공물에 금형을 대고 눌러서 원하는 형상을 만드는 기술로, 주로 자동차 부품 등 강 재료의 가공에 이용된다. 선반에 의한 절삭과 달리 절삭분이 나오지 않으며 형으로 프레스하여 동일한 형상의 부품을 대량 생산할 수 있다.
- 분석 대상 공정은 파인 블랭킹 프레스 가공으로서 판재를 잡아주는 클램핑의 힘과 위에서 아래로 찍는 펀치 힘, 아래서 위로 밀어주는 이젝터 힘으로 블랭크를 만든다. 재료에 V링 (V자 모양 홈)을 만들어 가공 중 재료가 움직이는 것을 막아 주는데, 이에 따라 블랭킹의 정밀도가 높아지고 표면이 깨끗하다.



[그림 1] 파인 블랭킹 프레스 공정

(출처 : <https://m.blog.naver.com/2xclub/221061835063>)

- 프레스 공정은 동력원으로서 유압펌프를 사용한다. 유압펌프는 외부에서 공급되는 기계적 에너지를 유압 시스템 작동유의 압력 에너지로 변환시켜주는 장치로서, 큰 힘을 내는데에 유리하여 많은 제조 공정에 사용되고 있다. 일반적으로, 회전하는 두 개의 기어 이빨 사이의 공간을 이용하여 가압하는 기어형(gear type), 회전하는 축에 설치된 베인을 이용하여 작동유를 밀어 압축시키는 베인형(vane type), 회전하는 축에 수직 또는 평행하게 설치된 여러 개의 실린더 안의 부피를 축의 회전방향에 따라 작아지게 만들어 압축하는 회전피스톤형(rotary piston type), 회전운동을 왕복운동으로 바꾸어 바로 실린더 안의 작동유를 피스톤으로 가압하는 왕복 피스톤형(reciprocating piston type) 등으로 나눌 수 있다.



[그림 2] 기어형 유압펌프 구조

(출처 : <https://terms.naver.com/entry.naver?docId=341371&cid=44616&categoryId=44616>)

- 유압펌프는 그 구조의 특성상, 작동 시 진동과 소음이 발생한다. 설비의 노후화로 인해 펌프 내부의 기어 마모가 발생하여 고장이 날 경우, 진동과 소음이 심해지며 출력 유압이 약해져 프레스 과정에 충분한 힘을 전달하지 못하게 된다. 이는 결국 불량품의 발생으로 이어진다.

2) 이슈사항(Pain point)

• 공정(설비)상의 문제 현황

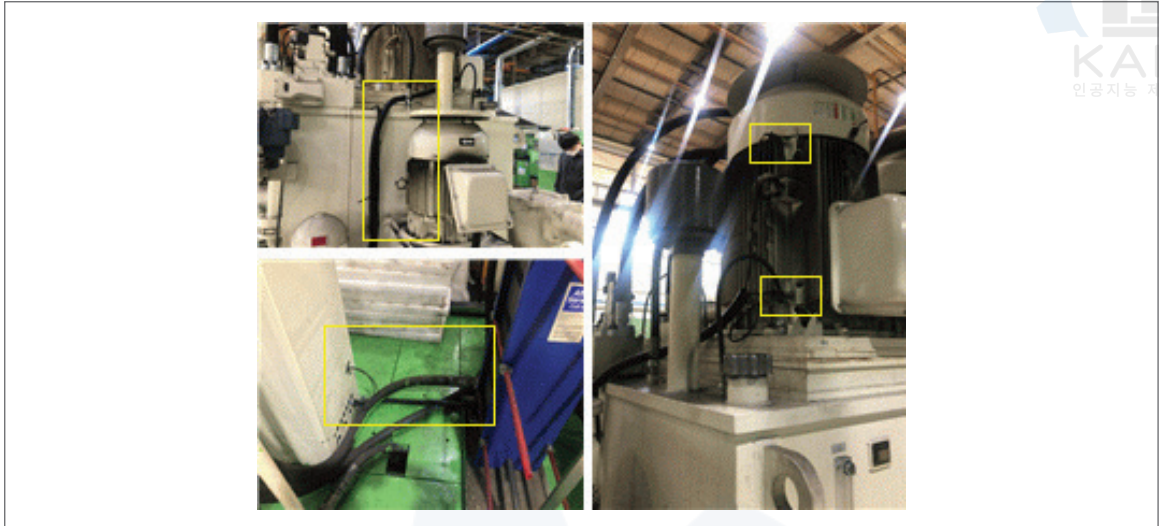
해당 공정은 파인 블랭킹 프레스 공정으로서 유압펌프를 사용하여 1000톤 가량의 힘을 프레스에 전달한다. 현재 설비의 노후화로 인해 펌프의 고장이 발생하면 공정 가동이 중단되어 생산계획에 차질이 발생한다. 펌프 고장시 수리비용 및 수리 부품의 수급 문제로 최대 2~3일 가량 가동이 중단되어 상당한 소요가 발생한다.

• 문제해결 장애요인

기존 유압펌프의 이상징후는 숙련된 작업자 혹은 관리자가 유압펌프 작동 소음으로 파악을 할 수 있었다. 또한, 설비 관련 데이터가 수집되지 않아 현황 모니터링이 불가하였다. 작업자의 경험에만 의존한 설비 관리가 이루어지고 있어 정확성이 떨어질 뿐만 아니라 숙련되지 않은 작업자의 경우에는 이상징후 파악이 불가능하다.

• 극복 방안

프레스 설비 유압펌프 상/하단에 진동센서를 설치하고 유압펌프 전력선에 전류 센서를 설치하여 데이터를 수집한다. 데이터는 시계열의 형태로 수집이 되므로, 시계열 이상 탐지 알고리즘을 사용하여 예지보전 모델을 개발한다. 모델은 정상작동 데이터로만 학습을 진행하여 이상 작동 데이터가 입력되었을 때 출력값의 오차가 크게 발생하게 되고, 이를 활용하여 이상 작동 데이터를 감지한다. 이를 활용하여 프레스 설비의 고장을 사전에 감지함으로써 극복이 가능하다.



[그림 3] 유압펌프 센서 설치 사진

1.2 분석 목표

1) 분석 목표

- 설비 센서 데이터(AI0_Vibration, AI2_Current 등)를 이용하여 AI를 통한 예지보전을 위한 분석을 진행한다.
- 시계열 딥러닝 알고리즘을 이용해 소성가공 공정에서의 예지보전 모델을 개발한다.
- 수집되는 데이터는 유압펌프 상부 진동, 유압펌프 하부 진동, 유압펌프 전류로 변수들 사이 상호성이 존재한다. 분석과정에서 이러한 특수성을 고려하여 이상 작동 여부를 복합적으로 감지하는 모델을 개발하여야 한다.

2) 데이터 정의 및 소개

- 프레스 설비 유압펌프에 설치된 센서를 통해 수집된 데이터를 사용한다.
- 주요 변수로는 AI0_Vibration(상부진동), AI1_Vibration(하부진동), AI2_Current(전류)가 있다.

3) 데이터 분석 기대효과

- 기존 공정의 예지보전 프로세스는 숙련된 작업자의 경험에만 의존하여 유압펌프의 고장을 효과적으로 감지할 수 없었다. 숙련자의 경우에도 정확한 감지를 할 수 없어 유압펌프의 고장을 방지할 수가 없었다.
- 본 분석에서는 정상 작동 상태의 유압펌프 진동 및 전류 데이터와 이상 작동 상태의 진동, 전류 데이터를 활용하여 시계열 이상 탐지 모델을 구축하고, 이를 활용하여 예지보



전을 진행한다. 이러한 데이터 기반의 예지보전 프로세스는 사람의 경험에 의존한 기존의 방식보다 신뢰성이 높을뿐더러, 시계열 데이터 및 알고리즘을 활용함으로써 미래의 고장을 예측할 수 있을 것으로 기대된다.

4) 시사점(implication) 요약기술

- 노후화된 소성가공 프레스 설비는 작업자가 그 이상징후를 파악하기 어려워 고장에 대비하기가 어렵다. 또한 고장 발생 후 수리까지 소요되는 시간 및 수리비용이 상당하여 생산계획에도 큰 차질이 발생한다. 본 데이터 분석을 통해 소성가공 설비에 예지보전 AI를 적용하여 생산성 향상과 비용 절감 등의 부분에 기여 할 수 있다.
- 또한, 유압펌프는 프레스 설비 뿐만 아니라 타 소성가공, 사출 등 여러 종류의 제조공정에서 설비에 동력을 전달하기 위해 사용되고 있다. 펌프의 노후화로 인해 내부 기어가 마모되어 고장이 발생하면, 동력 전달에 있어 손실이 발생하여 제품 생산에 악영향을 미친다. 본 분석 프로세스는 유압 펌프 외부에 설치된 센서로 펌프의 진동 및 전류 데이터를 수집하여 설비 예지보전 모델 구축이 가능하다. 이는 관련 인프라가 열악한 여러 중소기업의 공정에 확대 적용하기 용이하므로 시사하는 바가 크다.

2. 분석실습

2.1 제조데이터 소개

1) 데이터 수집 방법

- 소성가공 공정명 : 파인블랭킹 프레스
- 수집장비 : 프레스 설비 내 유압펌프 장착 센서
- 수집 기간 : 정상데이터 - 2022년 07월 12일, 이상데이터 - 2022년 07월 17일, 초당 10회 수집

2) 데이터 유형/구조

- 데이터셋 구조 : 시계열 형식(Time Series)
- 데이터 개수 :
(press_data_normal) column 수 5개, row 수 20,000개, 데이터수 100,000개
(outlier_data) column 수 5개, row 수 600개, 데이터수 3,000개
- 제조AI데이터셋 주요 변수 정의

속성(column)	설명	비고
TimeStamp	수집시간	object
AI0_Vibration	유압펌프 상부진동	float64
AI1_Vibration	유압펌프 하부진동	float64
AI2_Current	유압펌프 전류	float64
Equipment_state	장비 고장 여부	int64

[표 1] 데이터셋 주요 변수

• 주요 변수 기술 통계

속성(column)	데이터형	개수	평균	표준편차	최소값	중앙값	최대값
AI0_Vibration	float64	20000	-0.000094	0.070634	-0.314570	-0.000172	0.351699
AI1_Vibration	float64	20000	-0.000093	0.118888	-0.366498	-0.009317	0.399782
AI2_Current	float64	20000	1.446209	122.664494	-271.568330	1.849830	273.235000
Equipment_state	int64	20000	0.000000	0.000000	0.000000	0.000000	0.000000

[표 2] 정상 데이터 주요 변수 기술 통계

속성(column)	데이터형	개수	평균	표준편차	최소값	중앙값	최대값
AI0_Vibration	float64	600	0.007578	0.447928	-1.502735	0.015771	1.803204
AI1_Vibration	float64	600	-0.008137	0.241226	-0.751253	0.002161	0.509321
AI2_Current	float64	600	57.329741	153.172619	-399.351170	53.644187	538.826050
Equipment_state	int64	600	1.000000	0.000000	1.000000	1.000000	1.000000

[표 3] 이상 데이터 주요 변수 기술 통계

• 독립변수 / 종속변수

구분	명칭
독립변수	AI0_Vibration
	AI1_Vibration
	AI2_Current

- 본 분석에서는 LSTM-Autoencoder를 사용하여 이상치탐지를 진행한다. 장비 고장 상태를 나타내는 Equipment_state를 학습에 사용하지 않고 정상데이터와 이상데이터를 나누는 데에만 사용하여 종속변수를 기술하지 않는다.
- Equipment_state는 장비 고장상태를 나타내는 변수로서, 정상인 경우 0, 고장인 경우 1로 존재한다.

3) 데이터 (품질) 전처리

• 데이터 품질 전처리 목적

- 실제 공정에서 발생하는 데이터들은 값이 의미 없거나 누락 및 오타가 발생하여 품질이 떨어진다.
- 좋은 분석 방법론을 가지고 있더라도 품질이 낮은 데이터를 이용하면 좋은 결과를 얻을 수 없다.
- 그렇기에 데이터 (품질) 전처리는 데이터 분석에 있어서 필수적인 단계이다,
- 따라서 데이터들의 6가지 품질 지수를 파악하고, 데이터 전처리를 통해 품질 지수를 향상한다.

• 데이터 품질 지수

- 완전성(Completeness): 필수항목에 누락이 없어야 한다.
- 유일성(Uniqueness): 데이터 항목은 유일해야 하며 중복되어서는 안 된다.



- 유효성(Validity): 데이터 항목은 정해진 데이터 유효범위 및 도메인을 충족해야 한다.
- 일관성(Consistency): 데이터가 지켜야 할 구조, 값, 표현되는 형태가 일관되게 정의되고, 서로 일치해야 한다.
- 정확성(Accuracy): 실제 존재하는 객체의 표현 값이 정확하게 반영이 되어야 한다.
- 무결성(Integrity): 데이터베이스 자료의 오류 없이 변화에 영향을 받지 않고 데이터의 유일성, 유효성, 일관성이 보호되어야 한다.

• 데이터 품질 지수 세부 설명

- 완전성 품질 지수 = $(1 - (\text{결측치} / \text{전체 데이터 수})) * 100$

- ① 결측치 값이 30% 이상인 데이터들은 데이터의 완전성이 떨어지기 때문에 열별 결측치 값의 비율을 확인하여 삭제한다.
- ② 데이터의 결측치를 확인하기 위해 'isnull()' 함수를 사용한 뒤 'sum()' 함수를 이용하여 총 결측치 개수를 구한다.
- ③ 구한 결측치의 개수를 이용하여 완전성 품질 지수를 구한다.

- 유일성 품질 지수 = $((\text{유일한 데이터 수} / \text{전체 데이터 수})) * 100$

- ① 유일성을 판단하기 위해서는 유일한 값을 나타내는 열의 중복을 판단하면 된다.

- 유효성 품질 지수 = $(\text{유효성 만족 데이터 수} / \text{전체 데이터 수}) * 100$

- ① 데이터가 유효범위 내에 있는가?
- ② 데이터 형식에 맞는가?
- ③ 수집된 날짜 안에 들어가 있는가? 등을 검증하는 것이다.

- 일관성 품질 지수 = $(\text{일관성 만족 데이터 수} / \text{전체 데이터 수}) * 100$

- ① 데이터 테이블 간 종속관계 확인
- ② 데이터 형식에 맞는가?
- ③ 수집된 날짜 안에 들어가 있는가? 등을 검증하는 것이다.
데이터가 지켜야 할 구조, 값, 표현되는 형태가 일관되게 정의되고, 서로 일치해야 한다.

- 정확성 품질 지수 = $(1 - (\text{정확성 위배 데이터 수} / \text{전체 데이터 수})) * 100$

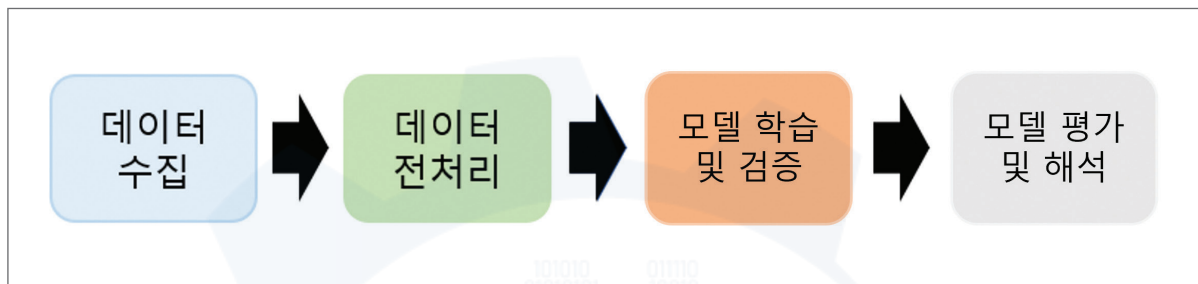
- 실제 존재하는 객체의 표현 값이 정확히 반영되어야 한다.

- 무결성 품질 지수 = $(1 - (\text{유일성, 유효성, 일관성 지수 중 100\%가 아닌 지수 개수} / 3)) * 100$

무결성 품질 지수는 고려되는 모든 데이터가 '유일성', '유효성', '일관성' 지수를 만족하는지 확인하는 지수이다. 본 가이드북에서는 세 가지 지수 중 100% 품질을 담당하는 지수의 비율을 무결성 품질 지수로 정의한다.

2.2 분석 모델 소개

1) 데이터 흐름 및 인공지능 모델 적용 흐름도



[그림 4] 데이터 흐름 및 인공지능 모델 적용 흐름도

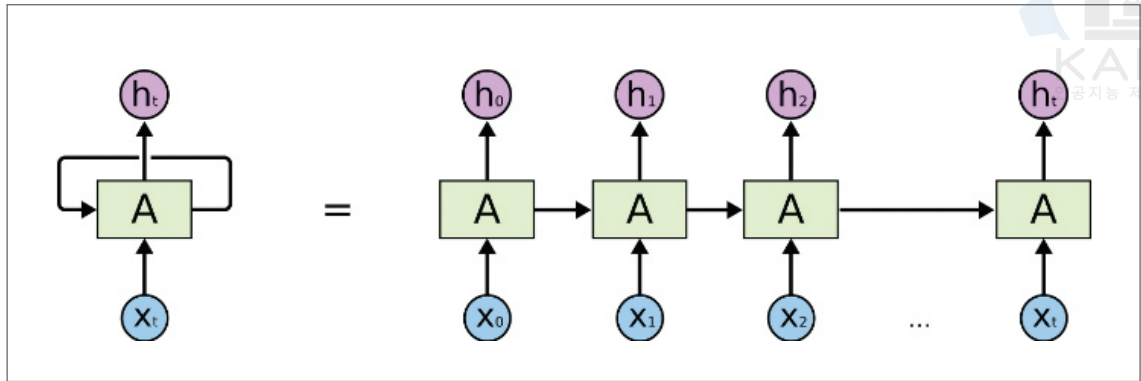
2) AI 분석모델

• 해당 AI 방법론(알고리즘) 선정 이유 기술

- 소성가공 예지보전 데이터셋은 프레스 설비 유압펌프에 부착된 센서를 통해 수집한 데이터로서 시계열의 형태를 가지고 있다. 이러한 시계열 데이터를 분석하기 위해서는 시간의 흐름을 고려하는 알고리즘을 사용하여야 한다.
- 일반적인 지도학습 형태의 설비 예지보전 모델을 개발하기 위해서는 충분한 양의 정상 데이터와 고장 데이터가 필요하다. 하지만 발생 빈도가 적은 고장 데이터의 특성상, 충분한 양의 고장 데이터를 수집할 수 없어 지도학습 알고리즘 사용이 불가능하다.
- LSTM-Autoencoder는 시간의 흐름을 고려하는 딥러닝 알고리즘인 LSTM과 이상치 탐지에 효과적인 비지도학습 알고리즘인 Autoencoder를 결합한 방법론으로서 본 분석에 적합하다.

• 적용하고자 하는 AI 분석 방법론(알고리즘)의 구체적 소개

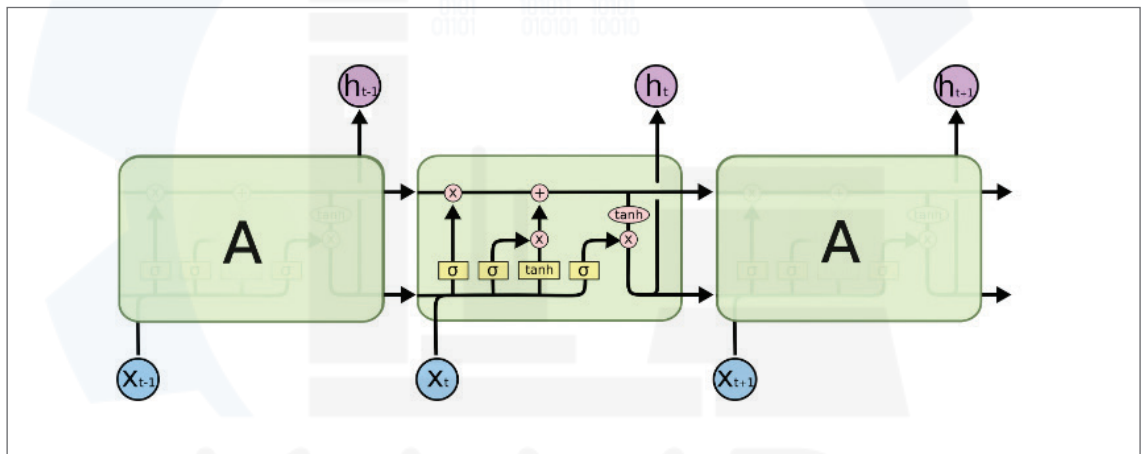
- LSTM(Long Short-Term Memory)은 RNN(Recurrent Neural Network)에서의 장기기억 소실 문제를 해결하기 위해 Cell State의 개념을 추가한 딥러닝 알고리즘이다. 전통적인 인공신경망 알고리즘은 이전에 일어난 사건의 영향을 고려할 수 없어 시간의 흐름에 영향을 받는 시계열 데이터에서 효과적으로 활용할 수 없었다. RNN의 순환 신경망 구조는 이러한 문제를 해결함으로써 시계열 데이터의 처리를 가능하게 하였다.



[그림 5] RNN 구조도

(출처 : <https://dgkim5360.tistory.com/entry/understanding-long-short-term-memory-lstm-kr>)

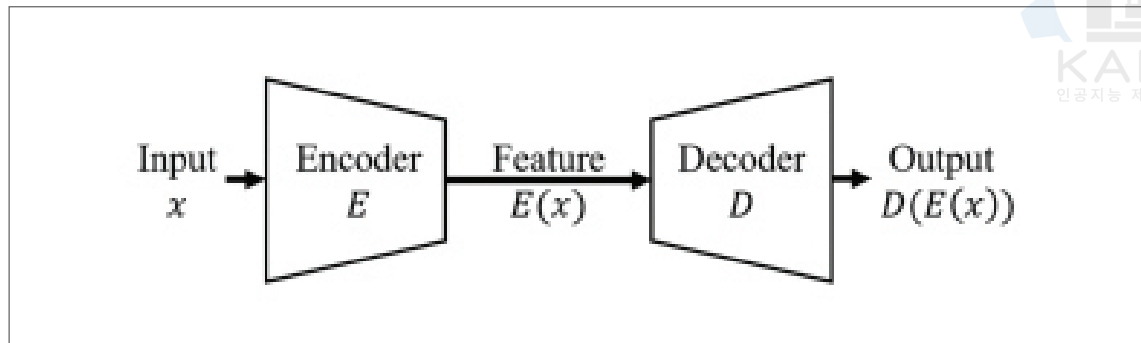
그러나 RNN은 장기기억 소실문제, 즉 현재 시점으로부터 먼 과거의 데이터를 활용하여야 할 때 성능이 저하되는 현상이 발생하였다. LSTM의 Cell State는 모델이 학습하는 과정에서 중요한 정보는 남기고, 중요하지 않은 정보는 버리는 과정을 가능하게 하여 장기기억 소실문제를 효과적으로 해결하였다.



[그림6] LSTM 구조도

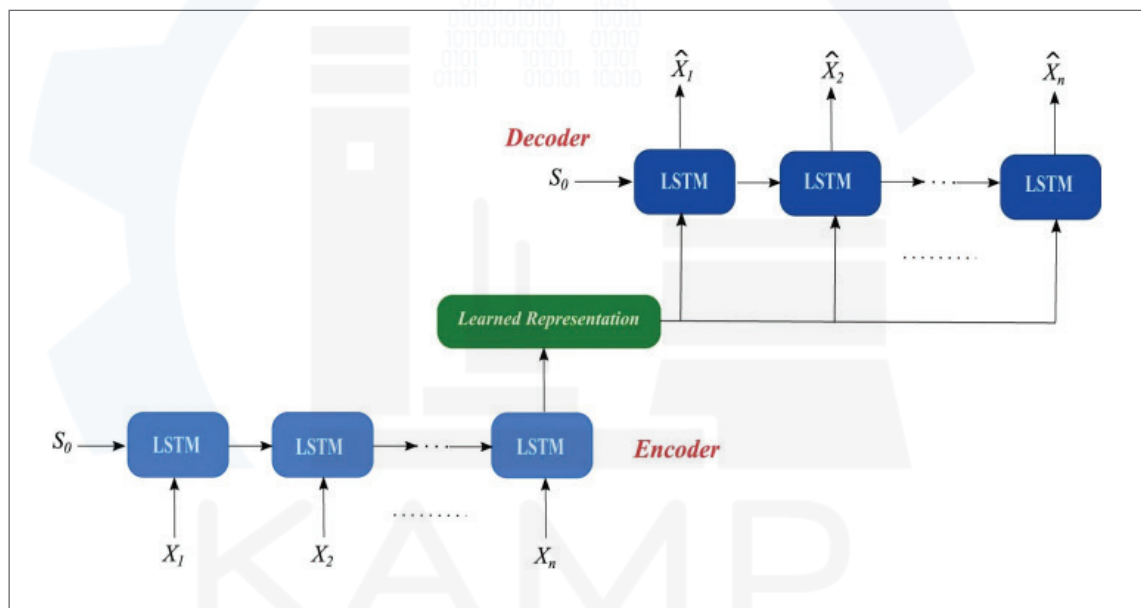
(출처 : <https://dgkim5360.tistory.com/entry/understanding-long-short-term-memory-lstm-kr>)

- Autoencoder는 입력 데이터를 압축하고 이를 원본에 가깝게 재구성하는 과정을 거쳐 데이터의 특징을 추출하는 대표적인 비지도 학습 방법론이다. Autoencoder는 입력데이터와 출력데이터의 오차 최소화를 목표로 학습을 하게된다. 이러한 성질을 바탕으로, 정상데이터로만 학습된 Autoencoder를 이상치 탐지에 사용한다. 학습된 Autoencoder에 정상데이터를 입력하면 출력 데이터의 재구성 오차 (Reconstruction Error)가 작게 나타나지만, 이상데이터를 입력하면 출력데이터의 재구성 오차가 크게 나타난다. 재구성 오차를 기반으로 적절한 임계값(Threshold)를 선정하여 데이터의 이상치 여부를 판별할 수 있다.



[그림 7] Autoencoder 구조도 (출처 : 김현재 외 6인(2021))

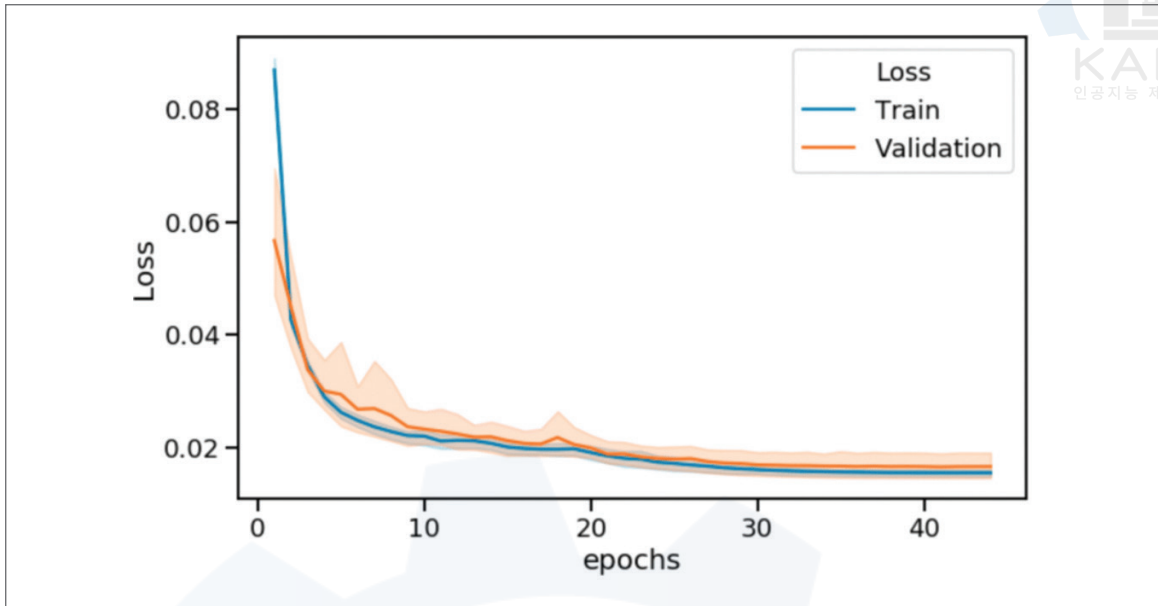
- LSTM-Autoencoder는 입력 데이터를 압축하고 이를 원본에 가깝게 재구성하는 과정을 거쳐 데이터의 특징을 추출하는 대표적인 비지도 학습 방법론인 Autoencoder에 시계열 데이터를 학습할 때 사용되는 LSTM 셀을 사용한 방법론으로, LSTM 셀을 적용한 인코더-디코더 구조로 구성되어 시계열 데이터에서의 이상치 탐지에 효과적이다.



[그림 8] LSTM-Autoencoder 구조도(출처 : 이종현 외 1명 (2021))

3) 학습 및 검증 절차

- 평균 제곱 오차(Mean Squared Error)는 알고리즘이 예측한 값과 실제 정답과의 차이인 오차를 제곱한 값의 평균이다. 평균 제곱 오차가 작을수록, 모델이 정답을 잘 맞힌다고 볼 수 있다. MSE 값이 작을수록 알고리즘의 성능이 높다. [그림7]과 같이 학습용 데이터와 검증용 데이터를 이용해 LSTM Autoencoder의 재구성 오차를 표현한다.

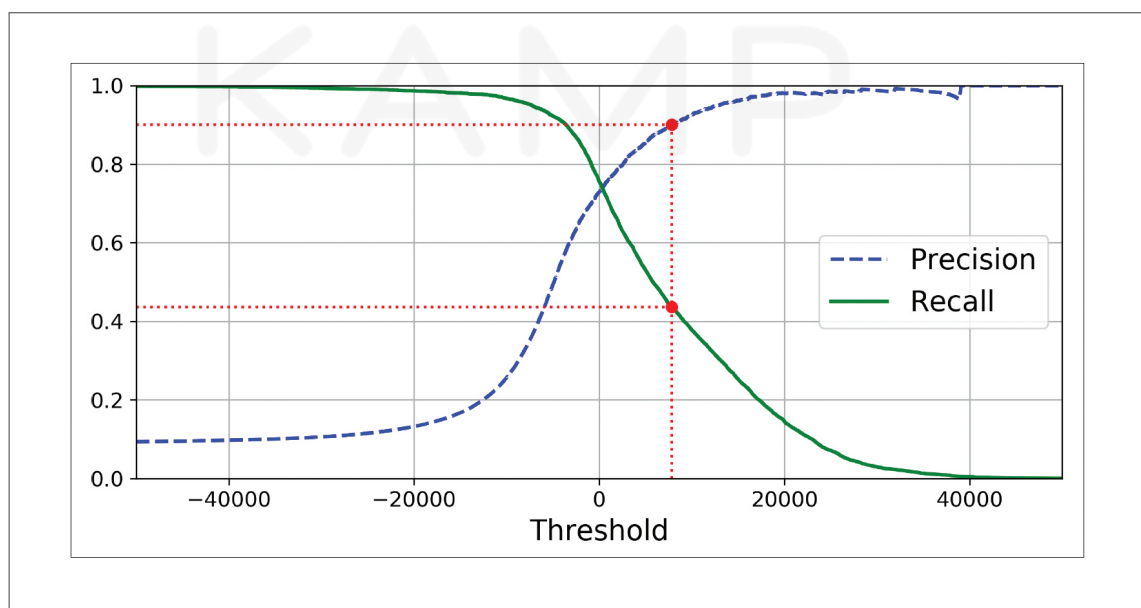


[그림 9] LSTM Autoencoder의 epoch 수에 따른 평균 제곱 오차

출처: https://www.researchgate.net/figure/Validation-loss-Autoencoder-training-and-validation-loss-during-training-epochs-after_fig5_337909876

• 정밀도-재현율 곡선(P-R Curve)

- P-R 곡선은 정밀도(precision)와 재현율(recall)의 관계를 나타내는 그래프다. 클래스 간 불균형이 심한 경우, 모델의 성능을 반영한다. 정상 설비 데이터와 이상 설비 데이터를 구분하기 위해 정밀도-재현율 곡선을 적용한다. 따라서 본 분석에서, 해당 곡선을 시각화하여 [그림 8]과 같이 정밀도 곡선과 재현율 곡선이 만나는 교점의 임계값을 정상 데이터와 이상 데이터를 식별하는 데 사용한다.

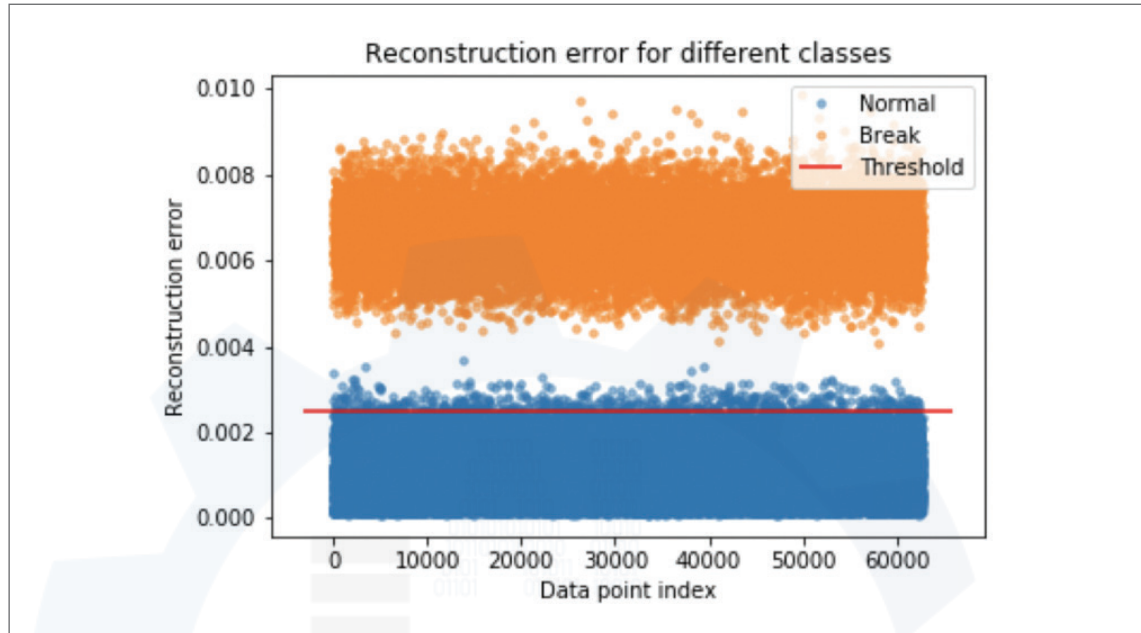


[그림 10] 정밀도-재현율 곡선



• 재구성 오차(Reconstruction Error)

- 학습된 LSTM Autoencoder에 테스트 데이터를 모델에 입력하여 평균 제곱 오차로 표현된 재구성 오차를 구한다. 이것을 정밀도-재현율 곡선을 통해 설정한 임계값과 비교하여 [그림 9]와 같이 정상 데이터와 이상 데이터를 구분할 수 있다.



[그림 11] 다른 클래스 데이터 간의 재구성 오차 시

출처: <https://blog.f-secure.com/process-creation-chains/>

• AI 분석 방법론(알고리즘) 구축 절차 설명

- 학습/검증/평가데이터 분할
 - 소성가공(프레스 설비) 데이터의 경우 정상 데이터와 이상 데이터가 구분되어있다. 모델 학습 시에는 정상 데이터만 사용하고, 검증 데이터와 평가 데이터에는 이상 데이터를 포함하여 학습된 모델이 이상치 탐지가 가능한지 평가한다.
- LSTM-Autoencoder 학습
 - LSTM-Autoencoder 알고리즘을 활용하여 입력 데이터를 압축하고 이를 원본에 가깝게 재구성하도록 학습을 진행한다. 이때 과적합이 발생하지 않도록 일정 epoch(학습 반복횟수)가 지나도 모델의 학습정도가 개선되지 않을 경우에 학습을 중단하는 조기종료(Early Stopping)를 적용한다.
- 모델 평가 및 해석
 - 검증용 데이터를 활용하여 이상치 탐지가 가능하도록 임계값을 정하고 시각화 한다. 이후 Confusion Matrix를 통해 모델의 성능을 평가하며, 정확도와 F1-score를 산출한다.

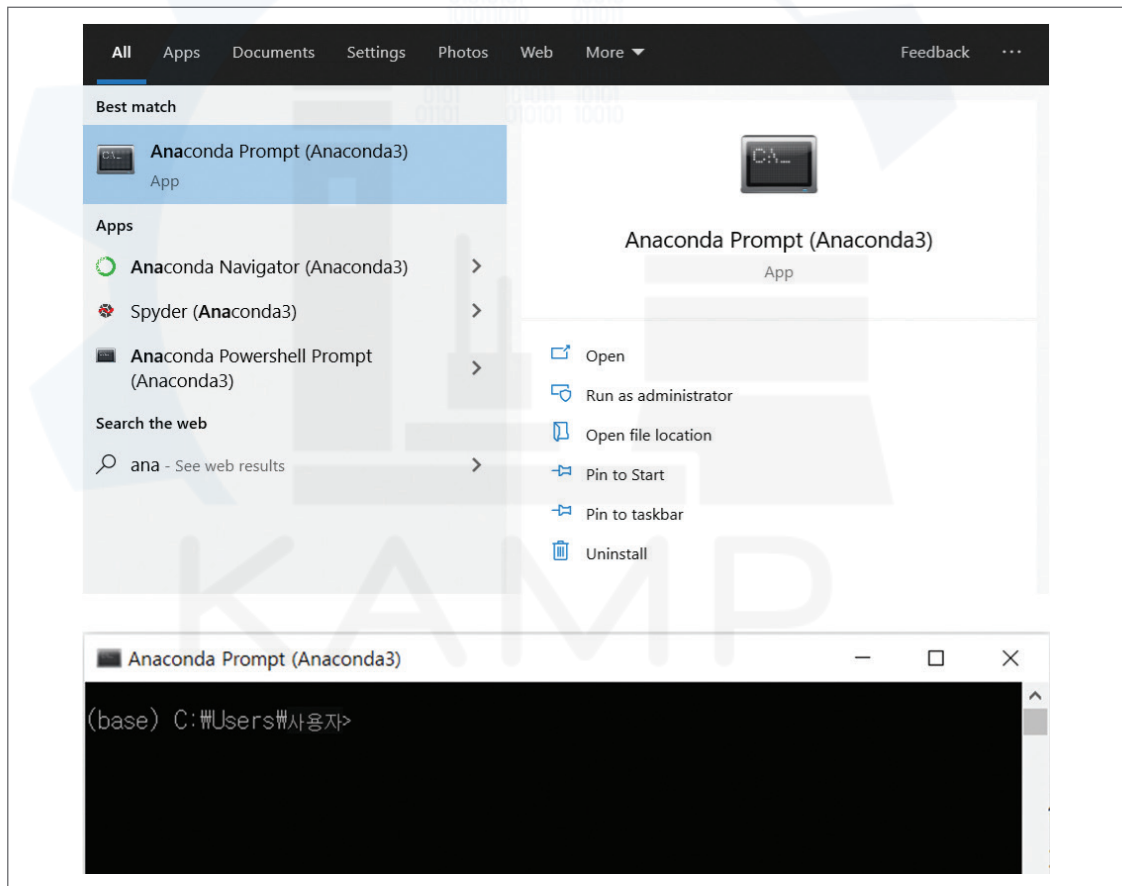
2.3 분석 체험

1) 필요 SW, 패키지 설치 방법 및 절차 가이드

- ▶ SW 설치는 '부록_분석환경 구축을 위한 설치 가이드' 참고
- ▶ 가상환경 구축 가이드

- 3. 부록 [분석 환경 구축을 위한 설치 가이드]에서 설치한 아나콘다(Anaconda)에서 파이썬의 독립적인 가상의 작업 환경을 구축한다. 파이썬의 모듈은 다양한 버전이 존재하기 때문에 충돌이 일어나기 쉬우며, 이러한 충돌을 방지하기 위해 실습마다 독립적인 가상 환경 구축을 권장한다. 가상 환경을 사용하기 위해서는 환경을 구축하고 실행(활성화)해야하고, 실습이 끝난 이후에는 환경을 비활성화 시켜야 한다. 주 피터 노트북에서 가상 환경을 실행하는 구체적인 순서 및 실행 방법은 아래와 같다.

- 아나콘다 프롬프트 실행





- 가상 환경 생성

```
# conda create -n 가상환경이름 python=버전
```

```
conda create -n KAMP python=3.9.7
```

```
Anaconda Prompt (anaconda3) - conda create -n KAMP python=3.9.7

(base) C:\Users\user>conda create -n KAMP python=3.9.7
Collecting package metadata (current_repodata.json): done
Solving environment: failed with repodata from current_repodata.json, will retry with next repodata source.
Collecting package metadata (repodata.json): done
Solving environment: done

## Package Plan ##

environment location: C:\Users\user\anaconda3\envs\KAMP
```

...

```
Proceed ([y]/n)? y # 해당 문구가 뜨면 'y' 를 입력한다.
```

...

```
done
#
# To activate this environment, use
#
#     $ conda activate KAMP
#
# To deactivate an active environment, use
#
#     $ conda deactivate
#
(base) C:\Users\user>
```

- 가상 환경 실행

가상 환경이 활성화되면 아래 그림처럼 명령 프롬프트 앞에 (가상 환경 이름)이 표시 된다.

```
# conda activate 가상환경이름
```

```
conda activate KAMP
```

```
(base) C:\Users\user>conda activate KAMP
(KAMP) C:\Users\user>
```

```
pip install ipykernel
```

```
# python -m ipykernel install --user --name 가상환경이름 --display-name "가상환경이름"
```

```
python -m ipykernel install --user --name KAMP --display-name "KAMP"
```

```
# jupyter notebook 실행
```

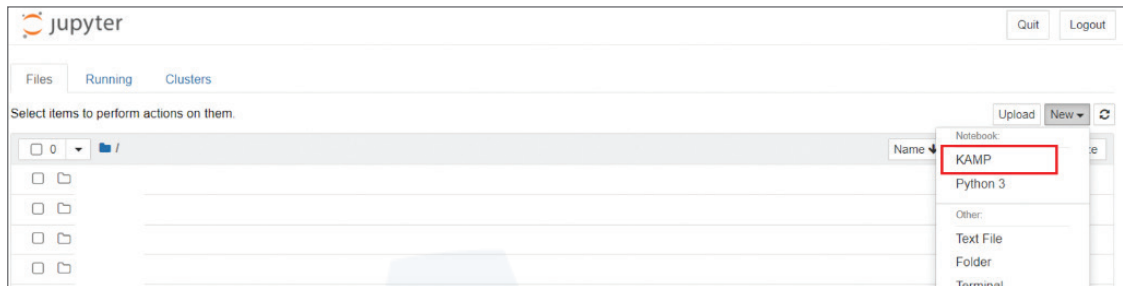
```
jupyter notebook
```

```
(KAMP) C:\Users\user>pip install ipykernel
Collecting ipykernel
  Using cached ipykernel-6.6.0-py3-none-any.whl (126 kB)
Collecting debugpy<2.0,>=1.0.0
  Using cached debugpy-1.5.1-cp37-cp37m-win_amd64.whl (4.3 MB)
Collecting matplotlib-inline<0.2.0,>=0.1.0
  Using cached matplotlib-inline-0.1.3-py3-none-any.whl (8.2 kB)
Collecting importlib-metadata<5
  Using cached importlib-metadata-4.8.2-py3-none-any.whl (17 kB)
Collecting tornado<7.0,>=4.2
  Using cached tornado-6.1-cp37-cp37m-win_amd64.whl (422 kB)
Collecting jupyter-client<8.0
  Using cached jupyter_client-7.1.0-py3-none-any.whl (129 kB)
Collecting ipython>=7.23.1
```

```
(KAMP) C:\Users\user>python -m ipykernel install --user --name KAMP --display-name "KAMP"
Installed kernelspec KAMP in C:\Users\user\AppData\Roaming\jupyter\kernels\kamp
```

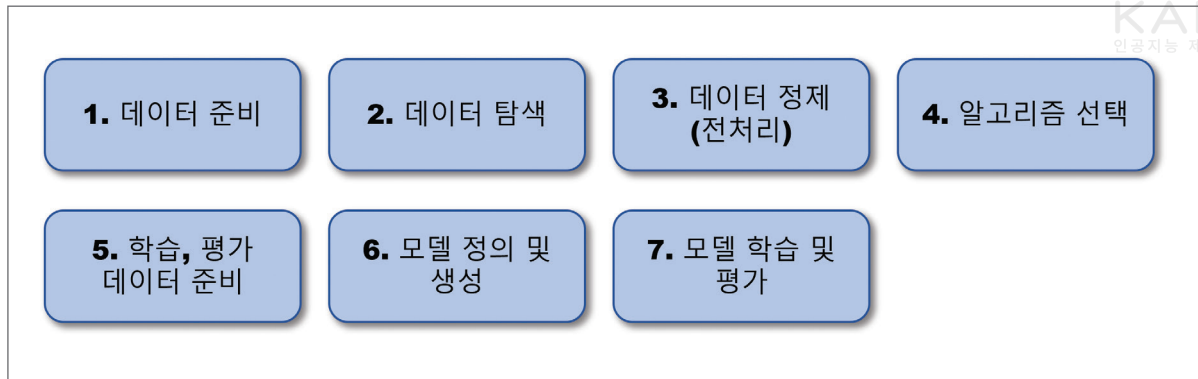
```
(KAMP) C:\Users\user>
```

```
(KAMP) C:\Users\User>jupyter notebook
[W 16:23:41.477 NotebookApp] All authentication is disabled. Anyone who can connect to this server will be able to run code.
[I 16:23:41.788 NotebookApp] JupyterLab extension loaded from C:\Users\User\Anaconda3\lib\site-packages\jupyterlab
[I 16:23:41.788 NotebookApp] JupyterLab application directory is C:\Users\User\Anaconda3\share\jupyter\lab
[I 16:23:41.790 NotebookApp] Serving notebooks from local directory: C:\Users\User\PycharmProjects\AIPoC
[I 16:23:41.790 NotebookApp] Jupyter Notebook 6.1.4 is running at:
[I 16:23:41.790 NotebookApp] http://localhost:8888/
[I 16:23:41.790 NotebookApp] Use Control-C to stop this server and shut down all kernels (twice to skip confirmation).
```





2) 분석 단계별 프로세스 - Flow Chart



[그림 8] Flow Chart

[단계 ①] 데이터 준비

①-1. 라이브러리/패키지 설치

```

!pip install numpy==1.20.0
!pip install pandas
!pip install seaborn
!pip install matplotlib
!pip install sklearn
!pip install tensorflow==2.7.0
  
```

```

Requirement already satisfied: scipy<1.0 in c:\users\user\anaconda3\lib\site-packages (from seaborn) (1.6.2)
Requirement already satisfied: numpy<1.15 in c:\users\user\anaconda3\lib\site-packages (from seaborn) (1.20.0)
Requirement already satisfied: pybars3<2.0.4,!=2.1.2,!=2.1.6,!=2.0.3 in c:\users\user\anaconda3\lib\site-packages (from matplotlib==2.2->seaborn) (2.4.7)
Requirement already satisfied: pillow<6.2.0 in c:\users\user\anaconda3\lib\site-packages (from matplotlib==2.2->seaborn) (8.2.0)
Requirement already satisfied: cycler<0.10 in c:\users\user\anaconda3\lib\site-packages (from matplotlib==2.2->seaborn) (0.10.0)
Requirement already satisfied: kiwisolver<1.0.1 in c:\users\user\anaconda3\lib\site-packages (from matplotlib==2.2->seaborn) (1.3.1)
Requirement already satisfied: python-dateutil<2.1 in c:\users\user\anaconda3\lib\site-packages (from matplotlib==2.2->seaborn) (2.8.1)
Requirement already satisfied: six in c:\users\user\anaconda3\lib\site-packages (from cycler<0.10->matplotlib==2.2->seaborn) (1.15.0)
Requirement already satisfied: pytz>2017.3 in c:\users\user\anaconda3\lib\site-packages (from pandas==0.23->seaborn) (2021.1)
Requirement already satisfied: matplotlib in c:\users\user\anaconda3\lib\site-packages (3.3.4)
Requirement already satisfied: pyparsing<2.0.4,!=2.1.2,!=2.1.6,!=2.0.3 in c:\users\user\anaconda3\lib\site-packages (from matplotlib==2.2->seaborn) (2.4.7)
Requirement already satisfied: pillow<6.2.0 in c:\users\user\anaconda3\lib\site-packages (from matplotlib==2.2->seaborn) (8.2.0)
Requirement already satisfied: cycler<0.10 in c:\users\user\anaconda3\lib\site-packages (from matplotlib==2.2->seaborn) (0.10.0)
Requirement already satisfied: python-dateutil<2.1 in c:\users\user\anaconda3\lib\site-packages (from matplotlib==2.2->seaborn) (2.8.1)
Requirement already satisfied: pyparsing<2.0.4,!=2.1.2,!=2.1.6,!=2.0.3 in c:\users\user\anaconda3\lib\site-packages (from matplotlib==2.2->seaborn) (2.4.7)
Requirement already satisfied: numpy<1.15 in c:\users\user\anaconda3\lib\site-packages (from matplotlib==2.2->seaborn) (1.20.0)
Requirement already satisfied: six in c:\users\user\anaconda3\lib\site-packages (from cycler<0.10->matplotlib==2.2->seaborn) (1.15.0)
Requirement already satisfied: sklearn in c:\users\user\anaconda3\lib\site-packages (0.0)
Requirement already satisfied: scikit-learn in c:\users\user\anaconda3\lib\site-packages (from sklearn) (1.1.2)
Requirement already satisfied: numpy<1.17.3 in c:\users\user\anaconda3\lib\site-packages (from scikit-learn->sklearn) (1.20.0)
Requirement already satisfied: threadpoolctl<2.0.0 in c:\users\user\anaconda3\lib\site-packages (from scikit-learn->sklearn) (2.1.0)
Requirement already satisfied: scipy<1.3.2 in c:\users\user\anaconda3\lib\site-packages (from scikit-learn->sklearn) (1.6.2)
Requirement already satisfied: joblib<0.10 in c:\users\user\anaconda3\lib\site-packages (from scikit-learn->sklearn) (1.2.0)
Requirement already satisfied: tensorflow==2.7.0 in c:\users\user\anaconda3\lib\site-packages (2.7.0)
Requirement already satisfied: google-pasta<0.1.1 in c:\users\user\anaconda3\lib\site-packages (from tensorflow==2.7.0) (0.2.0)
Requirement already satisfied: protobuf<3.9.2 in c:\users\user\anaconda3\lib\site-packages (from tensorflow==2.7.0) (3.17.2)
Requirement already satisfied: keras-preprocessing<1.1.1 in c:\users\user\anaconda3\lib\site-packages (from tensorflow==2.7.0) (1.1.2)
Requirement already satisfied: tensorflow==2.6 in c:\users\user\anaconda3\lib\site-packages (from tensorflow==2.7.0) (2.11.0)
Requirement already satisfied: gast<0.5.0,>=0.2.1 in c:\users\user\anaconda3\lib\site-packages (from tensorflow==2.7.0) (0.4.0)
Requirement already satisfied: flatbuffers<3.0,>=1.12 in c:\users\user\anaconda3\lib\site-packages (from tensorflow==2.7.0) (2.0.7)
Requirement already satisfied: libclang<9.0.1 in c:\users\user\anaconda3\lib\site-packages (from tensorflow==2.7.0) (14.0.6)
Requirement already satisfied: astunparse<1.6.0 in c:\users\user\anaconda3\lib\site-packages (from tensorflow==2.7.0) (1.6.3)
Requirement already satisfied: keras<2.8,>=2.7.0rc0 in c:\users\user\anaconda3\lib\site-packages (from tensorflow==2.7.0) (2.7.0)
Requirement already satisfied: tensorflow-estimator<2.8,>=2.7.0rc0 in c:\users\user\anaconda3\lib\site-packages (from tensorflow==2.7.0) (2.7.0)
Requirement already satisfied: tensorflow-io-gcs-filesystem<0.21.0 in c:\users\user\anaconda3\lib\site-packages (from tensorflow==2.7.0) (0.27.0)
Requirement already satisfied: six<1.12.0 in c:\users\user\anaconda3\lib\site-packages (from tensorflow==2.7.0) (1.15.0)
Requirement already satisfied: termcolor<1.1.0 in c:\users\user\anaconda3\lib\site-packages (from tensorflow==2.7.0) (1.1.0)
Requirement already satisfied: wrapt<1.11.0 in c:\users\user\anaconda3\lib\site-packages (from tensorflow==2.7.0) (1.12.1)
Requirement already satisfied: h5py<2.9.0 in c:\users\user\anaconda3\lib\site-packages (from tensorflow==2.7.0) (2.10.0)
Requirement already satisfied: absl-py<0.4.0 in c:\users\user\anaconda3\lib\site-packages (from tensorflow==2.7.0) (0.13.0)
Requirement already satisfied: numpy<1.14.5 in c:\users\user\anaconda3\lib\site-packages (from tensorflow==2.7.0) (1.20.0)
Requirement already satisfied: typing-extensions<3.6.6 in c:\users\user\anaconda3\lib\site-packages (from tensorflow==2.7.0) (3.7.4.3)
Requirement already satisfied: grpcio<2.0,>=1.24.3 in c:\users\user\anaconda3\lib\site-packages (from tensorflow==2.7.0) (1.36.1)
Requirement already satisfied: opt-einsum<2.3.2 in c:\users\user\anaconda3\lib\site-packages (from tensorflow==2.7.0) (3.3.0)
Requirement already satisfied: tensorflow-plugin-interface<1.6.0 in c:\users\user\anaconda3\lib\site-packages (from tensorflow==2.6->tensorflow==2.7.0) (1.6.0)
Requirement already satisfied: wheel<1.0,>=0.32.0 in c:\users\user\anaconda3\lib\site-packages (from tensorflow==2.7.0) (0.36.2)
Requirement already satisfied: tensorflow-data-server<0.7.0,>=0.6.0 in c:\users\user\anaconda3\lib\site-packages (from tensorflow==2.6->tensorflow==2.7.0) (0.6.1)
Requirement already satisfied: cachetools<5.0,>=2.0.0 in c:\users\user\anaconda3\lib\site-packages (from google-auth<3,>=1.6.3->tensorflow==2.6->tensorflow==2.7.0) (4.2.2)
Requirement already satisfied: rsa<5,>=3.1.4 in c:\users\user\anaconda3\lib\site-packages (from google-auth<3,>=1.6.3->tensorflow==2.6->tensorflow==2.7.0) (4.7.2)
Requirement already satisfied: pyasn1-modules<0.2.1 in c:\users\user\anaconda3\lib\site-packages (from google-auth<3,>=1.6.3->tensorflow==2.6->tensorflow==2.7.0) (0.2.8)
Requirement already satisfied: requests-oauthlib<0.7.0 in c:\users\user\anaconda3\lib\site-packages (from google-auth-oauthlib<0.5,>=0.4.1->tensorflow==2.6->tensorflow==2.7.0) (1.3.0)
Requirement already satisfied: pyasn1<0.5.0,>=0.4.6 in c:\users\user\anaconda3\lib\site-packages (from pyasn1-modules==0.2.1->google-auth<3,>=1.6.3->tensorflow==2.6->tensorflow==2.7.0) (0.4.8)
Requirement already satisfied: idna<3,>=2.5 in c:\users\user\anaconda3\lib\site-packages (from requests<3,>=2.21.0->tensorflow==2.6->tensorflow==2.7.0) (2.10)
Requirement already satisfied: certifi<2017.4.17 in c:\users\user\anaconda3\lib\site-packages (from requests<3,>=2.21.0->tensorflow==2.6->tensorflow==2.7.0) (2020.12.5)
Requirement already satisfied: chardet<5,>=3.0.2 in c:\users\user\anaconda3\lib\site-packages (from requests<3,>=2.21.0->tensorflow==2.6->tensorflow==2.7.0) (4.0.0)
Requirement already satisfied: urllib3<1.27,>=1.21.1 in c:\users\user\anaconda3\lib\site-packages (from requests<3,>=2.21.0->tensorflow==2.6->tensorflow==2.7.0) (1.26.4)
Requirement already satisfied: oauthlib<3.0.0 in c:\users\user\anaconda3\lib\site-packages (from requests-oauthlib==0.7.0->google-auth-oauthlib<0.5,>=0.4.1->tensorflow==2.6->tensorflow==2.7.0) (3.1.1)
  
```

[코드 1] 라이브러리/패키지 설치



- 분석에 필요한 Library 설치를 위해 코드 1과 같이 설치를 진행한다. 코드 작성 후 ▶ Run 버튼이나 shift + enter를 눌러 실행시킨다.

①-2. 라이브러리/패키지 불러오기

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
import seaborn as sns
import tensorflow as tf
from tensorflow.keras import Model, models, layers, optimizers, regularizers
from sklearn.preprocessing import MinMaxScaler
from sklearn import metrics
from sklearn.model_selection import train_test_split
```

[코드 2] Library import

- 코드 2를 통해 분석에 필요한 Library를 불러온다.
- Library를 불러오는 코드는 'import Library'를 기본으로 한다.
- Library를 불러와서 다른 이름(alias)으로 지정하는 코드는 'import Library as alias'를 기본으로 한다.
- Library 내 Function을 불러오는 코드는 'from Library import Function' 또는 'import Library.Function'을 기본으로 한다. 첫 번째 방법을 사용 시, Function을 사용할 때, Function만 기입하여 사용하면 되며, 두 번째 방법을 사용 시, Function을 사용할 때, Library.Function 형태로 사용해야 한다.
- Library 내 Function을 불러와서 다른 이름(alias)으로 지정하는 코드는 'from Library import Function as alias'를 기본으로 한다.

①-3. 데이터 불러오기

```
# 데이터 불러오기

normal = pd.read_csv('./press_data_normal.csv', index_col=0)
outlier = pd.read_csv('./outlier_data.csv', index_col=0)
```

[코드 3] 데이터 로드 및 출력 코드

- 코드 3을 통해 프레스 설비 정상작동 데이터와 이상작동 데이터를 불러온다.



정상 데이터 상위 5개의 행 출력

```
normal.head()
```

	TimeStamp	AI0_Vibration	AI1_Vibration	AI2_Current	Equipment_state
0	2022-07-12 00:00:00.019	0.116967	-0.176403	192.338730	0
1	2022-07-12 00:00:00.119	0.030080	-0.210895	217.004330	0
2	2022-07-12 00:00:00.219	-0.045794	-0.029424	197.510620	0
3	2022-07-12 00:00:00.319	0.047519	0.258642	156.607170	0
4	2022-07-12 00:00:00.419	0.020351	0.173777	93.236934	0

[코드 4] 상위 5개의 행 출력 코드

- 코드 4을 통해 정상 데이터프레임의 상위 5개의 행을 출력한다.

이상 데이터 상위 5개의 행 출력

```
outlier.head()
```

	TimeStamp	AI0_Vibration	AI1_Vibration	AI2_Current	Equipment_state
0	2022-07-17 10:51:07.943	0.293727	-0.540554	21.457675	1
1	2022-07-17 10:51:08.043	1.070320	-0.557382	-44.107442	1
2	2022-07-17 10:51:08.143	0.908692	0.063108	25.033954	1
3	2022-07-17 10:51:08.243	0.235921	0.509321	-101.327910	1
4	2022-07-17 10:51:16.724	0.315538	0.205315	34.570698	1

[코드 5] 하위 5개의 행 출력 코드

- 코드 5를 통해 이상 데이터프레임의 상위 5개의 행을 출력한다.

[단계 ②] 데이터 탐색

②-1. 데이터 사본 생성

데이터 사본 생성

```
normal_data = normal.copy()
outlier_data = outlier.copy()
```

[코드 6] 데이터의 사본 생성 코드

- 코드 6에서 정상 데이터프레임과 이상 데이터프레임의 사본을 copy()를 통해 생성한다.



②-2. 데이터 수 확인

정상 데이터, 이상 데이터 개수 확인

```
print('Normal Data 개수 : {}'.format(len(normal_data)))
print('Outlier Data 개수 : {}'.format(len(outlier_data)))
```

```
Normal Data 개수 : 20000
Outlier Data 개수 : 600
```

[코드 7] 데이터 칼럼별 통계 확인 코드

- 코드 7에서 정상데이터와 이상데이터의 개수를 확인한다.

②-3. 정상/이상 데이터 기초 통계량 확인

정상 데이터 기초통계량 확인

```
normal_data.describe().T
```

	count	mean	std	min	25%	50%	75%	max
AI0_Vibration	20000.0	-0.000094	0.070634	-0.314570	-0.044168	-0.000171	0.044011	0.351699
AI1_Vibration	20000.0	-0.000093	0.118888	-0.366498	-0.069247	-0.009317	0.060176	0.399782
AI2_Current	20000.0	1.446209	122.664494	-271.568330	-101.311693	1.849830	103.844418	273.235000
Equipment_state	20000.0	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000

[코드 8] 데이터 칼럼별 타입 확인

- 코드 8을 통해 정상데이터의 기초통계량을 확인한다.

이상 데이터 기초통계량 확인

```
outlier_data.describe().T
```

	count	mean	std	min	25%	50%	75%	max
AI0_Vibration	600.0	0.007578	0.447928	-1.502735	-0.116400	0.015771	0.092859	1.803204
AI1_Vibration	600.0	-0.008137	0.241226	-0.751253	-0.136426	0.002161	0.171432	0.509321
AI2_Current	600.0	57.329741	153.172619	-399.351170	-33.378605	53.644187	146.627440	538.826050
Equipment_state	600.0	1.000000	0.000000	1.000000	1.000000	1.000000	1.000000	1.000000

[코드 9] 데이터 크기 확인 코드

- 코드 9를 통해 정상데이터의 기초통계량을 확인한다.

- describe 함수를 사용하여 데이터프레임의 기초통계량을 확인할 수 있다.



②-4. 데이터프레임 null 값 개수 확인

정상 데이터 결측치 개수 파악

```
normal_data.isnull().sum()
```

```
TimeStamp      0
AI0_Vibration  0
AI1_Vibration  0
AI2_Current    0
Equipment_state 0
dtype: int64
```

[코드 10] 데이터프레임 null 값 개수 확인 코드

이상 데이터 결측치 개수 파악

```
outlier_data.isnull().sum()
```

```
TimeStamp      0
AI0_Vibration  0
AI1_Vibration  0
AI2_Current    0
Equipment_state 0
dtype: int64
```

[코드 11] 데이터프레임 null 값 개수 확인 코드

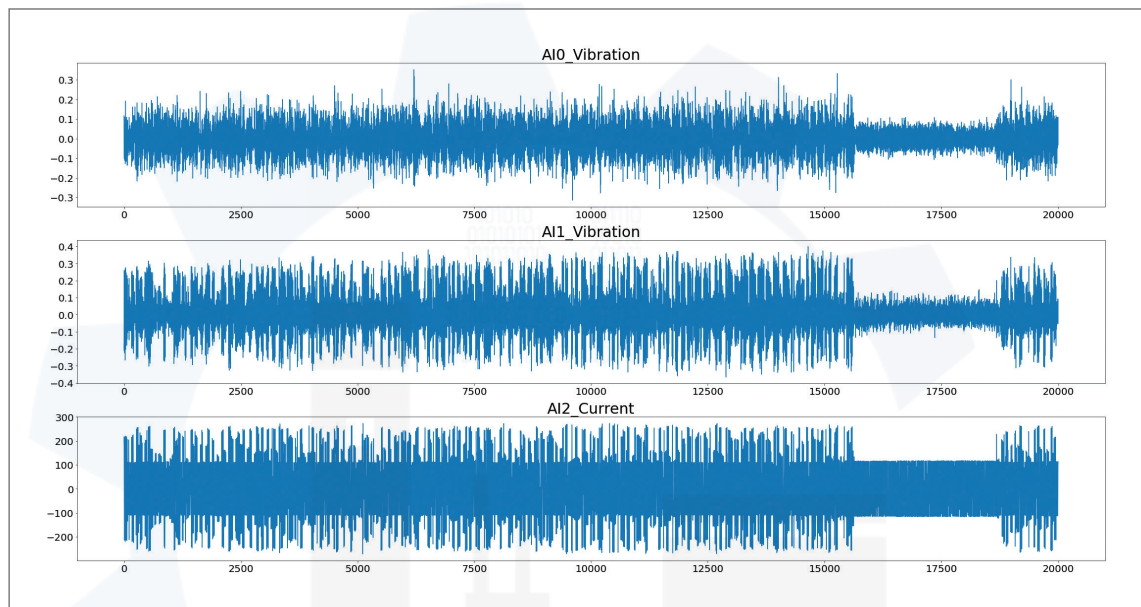
- 코드 10와 코드 11으로 변수별 결측 개수를 확인할 수 있으며, isnull()은 각 변수에서 결측이 있다면 True를 반환하고 True 개수를 sum()을 이용해 합하여 출력한다.

②-5. 정상/이상 데이터 시각화

정상데이터 시각화

```
use_col = ['AI0_Vibration', 'AI1_Vibration', 'AI2_Current']
plt.figure(figsize=(30,15))
for i in range(len(use_col)):
    plt.subplot(3, 1, i+1)
    plt.title(use_col[i], fontsize=30)
    plt.plot(normal_data[use_col[i]])
    plt.xticks(size=20)
    plt.yticks(size=20)

plt.tight_layout()
```



[코드 12] 정상 데이터 시각화

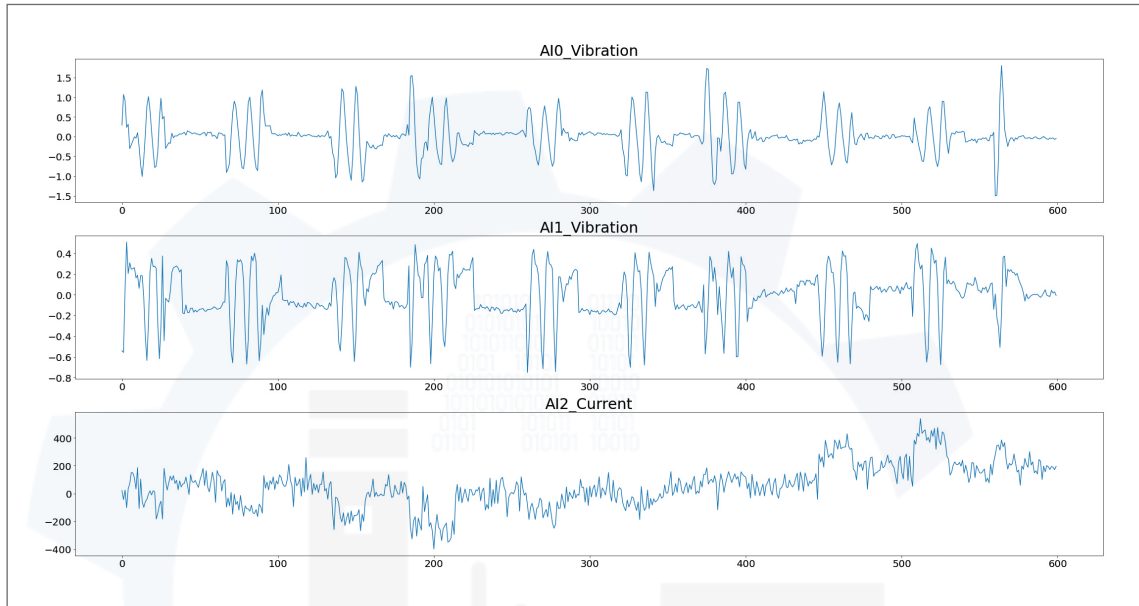
- 코드 12를 통해 정상데이터를 시각화한다.



이상 데이터 시각화

```
use_col = ['AI0_Vibration', 'AI1_Vibration', 'AI2_Current']
plt.figure(figsize=(30,15))
for i in range(len(use_col)):
    plt.subplot(3, 1, i+1)
    plt.title(use_col[i], fontsize=30)
    plt.plot(outlier_data[use_col[i]])
    plt.xticks(size=20)
    plt.yticks(size=20)

plt.tight_layout()
```



[코드 13] 이상 데이터 시각화

- 코드 13을 통해 이상데이터를 시각화 한다.
- 소성가공 예지보전 데이터셋의 경우, 데이터가 시계열로 존재하여 plt.plot함수를 사용하여 시간에 따른 데이터의 변화를 시각화한다.

[단계 ③] 데이터 정제

③-1. 데이터 전처리

진동 형태의 정상/이상 데이터를 절댓값을 취해 진폭으로 변환

```
normal_data[use_col] = normal_data[use_col].applymap(lambda x: abs(x))
outlier_data[use_col] = outlier_data[use_col].applymap(lambda x: abs(x))
```

[코드 14] 숫자형 변수 데이터 재구성 코드

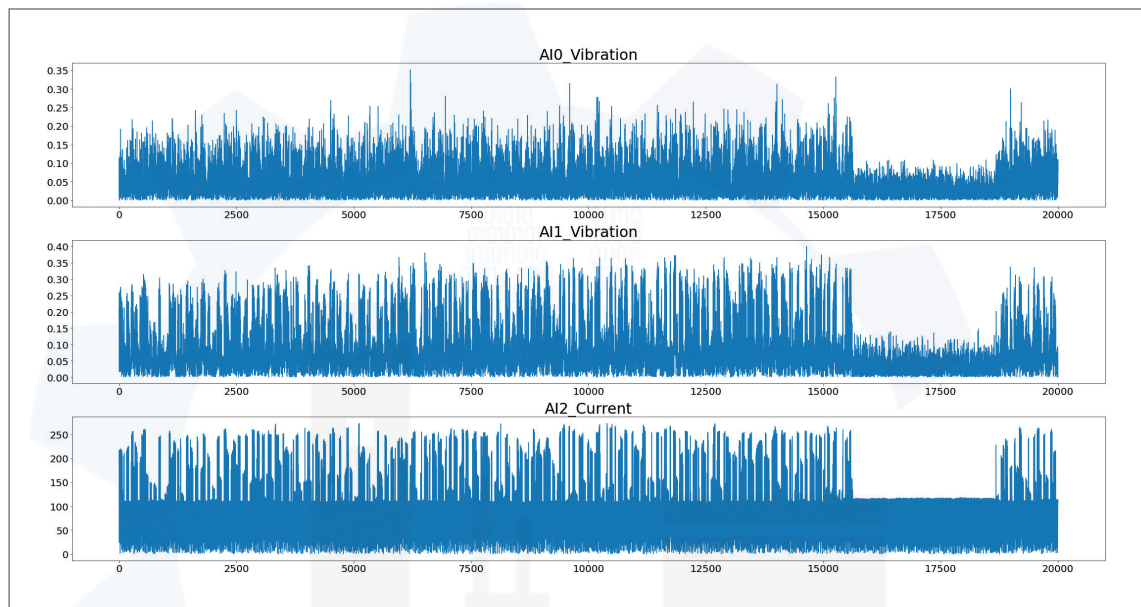
- 코드 14을 통해 진동 형태의 원본데이터에 절댓값을 취해 진폭 형태로 변환한다.
- applymap 함수와 lambda 함수를 사용하여 데이터프레임의 각 원소마다 절댓값 함수를 적용할 수 있다.

③-2. 전처리 후 데이터 시각화

전처리 후 정상 데이터 시각화

```
use_col = ['AI0_Vibration', 'AI1_Vibration', 'AI2_Current']
plt.figure(figsize=(30,15))
for i in range(len(use_col)):
    plt.subplot(3, 1, i+1)
    plt.title(use_col[i], fontsize=30)
    plt.plot(normal_data[use_col[i]])
    plt.xticks(size=20)
    plt.yticks(size=20)

plt.tight_layout()
```



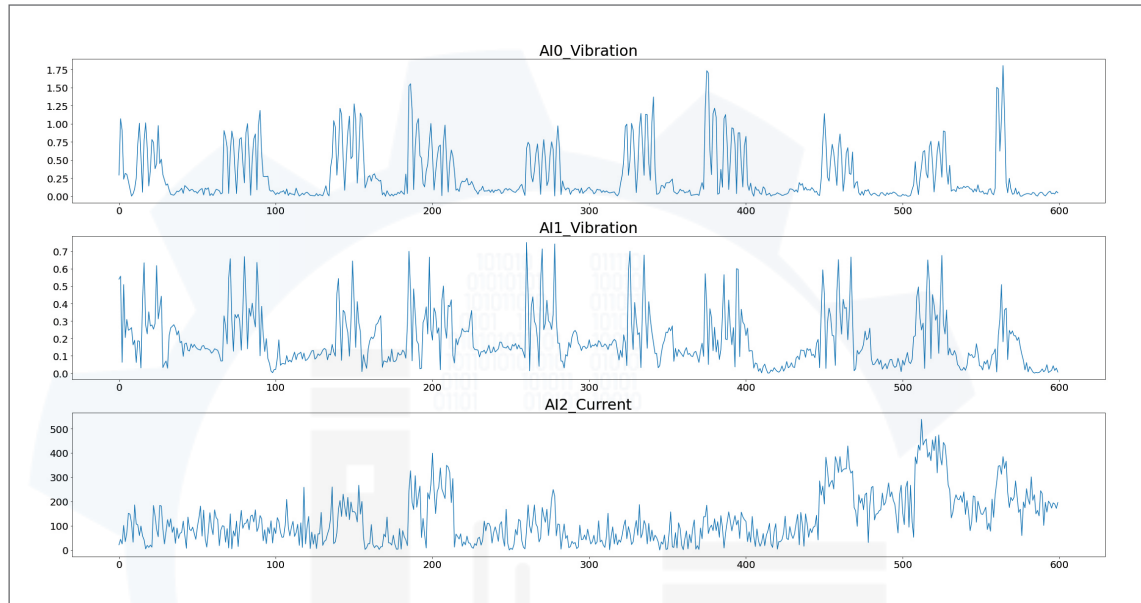
[코드 15] 전처리 후 정상 데이터 시각화 코드

- 코드 15를 통해 전처리 후의 정상 데이터를 시각화한다.

전처리 후 이상 데이터 시각화

```
use_col = ['AI0_Vibration', 'AI1_Vibration', 'AI2_Current']
plt.figure(figsize=(30,15))
for i in range(len(use_col)):
    plt.subplot(3, 1, i+1)
    plt.title(use_col[i], fontsize=30)
    plt.plot(outlier_data[use_col[i]])
    plt.xticks(size=20)
    plt.yticks(size=20)

plt.tight_layout()
```



[코드 16] 전처리 후 이상 데이터 시각화 코드

- 코드 16을 통해 전처리 후의 이상 데이터를 시각화한다.

③-3. 최종 데이터 상관관계 분석

최종 정상 데이터 상관관계 분석

```
normal_data[use_col].corr()
```

	AI0_Vibration	AI1_Vibration	AI2_Current
AI0_Vibration	1.000000	0.220893	0.136037
AI1_Vibration	0.220893	1.000000	0.324596
AI2_Current	0.136037	0.324596	1.000000

[코드 17] 최종 변수별 상관관계 확인 코드



최종 이상 데이터 상관관계 분석

```
outlier_data[use_col].corr()
```

	AI0_Vibration	AI1_Vibration	AI2_Current
AI0_Vibration	1.000000	0.355706	0.243357
AI1_Vibration	0.355706	1.000000	0.214997
AI2_Current	0.243357	0.214997	1.000000

[코드 18] 최종 변수별 상관관계 확인 코드

- 코드 17, 18을 통해 정상 데이터/이상 데이터 변수 간 상관관계를 확인할 수 있다.
- 정상/이상 데이터에서 변수 간 양의 상관관계를 확인 할 수 있다.

[단계 ④] 알고리즘 선택

④-1. 시계열 이상탐지 알고리즘

- 프레스 설비 이상 작동 탐지를 위한 모델은 LSTM-Autoencoder로 이상치 탐지에 유효한 Autoencoder와 시계열 데이터에 적합한 LSTM을 결합한 모델을 사용한다.
- LSTM-Autoencoder 모델은 정상 작동 시계열 데이터로만 학습을 진행하고, 학습한 데이터 분포에서 벗어난 데이터를 이상치로 판정한다.

[단계 ⑤] 학습, 평가 데이터 준비

⑤-1. 학습 데이터, 평가 데이터 분리

정상/이상 데이터 준비

```
X_normal = normal_data[use_col]
y_normal = normal_data["Equipment_state"]
X_anomaly = outlier_data[use_col]
y_anomaly = outlier_data["Equipment_state"]
```

학습 및 평가 데이터 분리

```
X_train_normal = X_normal[:15000]
y_train_normal = y_normal[:15000]
X_test_normal = X_normal[15000:]
y_test_normal = y_normal[15000:]
X_test_anomaly = X_anomaly
y_test_anomaly = y_anomaly
```

[코드 19] 모델 학습을 위한 Train, Test 셋 분리 코드



- 코드 19를 통해 학습용 데이터, 평가용 데이터를 구분한다.
- 해당 데이터는 시계열 데이터로서, 모델이 학습하지 않은 미래의 데이터에 유효하여야 한다. 이를 위해 15000번째 이후의 정상데이터와 전체 이상 데이터를 평가용 데이터로 사용한다.

⑤-2. 최대-최소 정규화 적용

```
#정규화 진행

scaler = MinMaxScaler()

X_train_scaled = scaler.fit_transform(X_train_normal)
X_test_normal_scaled = scaler.transform(X_test_normal)
X_test_anomaly_scaled = scaler.transform(X_test_anomaly)

y_train_normal = np.array(y_train_normal)
y_test_normal = np.array(y_test_normal)
y_test_anomaly = np.array(y_test_anomaly)
```

[코드 20] 최대-최소 정규화 적용 코드

- 코드 20을 통해 최대-최소 정규화를 적용한다.
- 전체 데이터셋에 정규화를 적용 시, 학습용 데이터셋이 평가용 데이터셋에 영향을 받게되어 왜곡된 모델이 생성될 수 있으므로, 학습용 데이터에만 스케일러의 fit_transform()함수를 적용하고, 평가용 데이터에는 transform()함수만 적용한다.



⑤-3. LSTM 학습을 위한 shape 변경

```
# LSTM 학습을 위한 구조 변경

sequence = 20
X_train, Y_train = [], []
for index in range(len(X_train_scaled) - sequence - 100):
    X_train.append(X_train_scaled[index: index + sequence])
    Y_train.append(y_train_normal[index + sequence + 100])

#평가용 정상 데이터

X_test_normal, Y_test_normal = [], []
for index in range(len(X_test_normal_scaled) - sequence - 100):
    X_test_normal.append(X_test_normal_scaled[index: index + sequence])
    Y_test_normal.append(y_test_normal[index + sequence + 100])

#평가용 이상 데이터

X_test_anomal, Y_test_anomal = [], []
for index in range(len(X_test_anomaly_scaled) - sequence - 100):
    X_test_anomal.append(X_test_anomaly_scaled[index: index + sequence])
    Y_test_anomal.append(y_test_anomal[index + sequence + 100])

#리스트 형태를 numpy array로 변환

X_test_normal, Y_test_normal = np.array(X_test_normal), np.array(Y_test_normal)
X_test_anomal, Y_test_anomal = np.array(X_test_anomal), np.array(Y_test_anomal)
```

[코드 21] LSTM용 데이터 구조변경 코드

- LSTM셀은 3차원 데이터를 입력받으므로, 코드 21을 통해 데이터의 구조를 변경하여 LSTM-Autoencoder 학습을 가능케 한다.
- 데이터의 구조는 (n, sequence, m)으로, n은 시계열 데이터의 길이, sequence는 모델이 다음 시점의 미래 예측을 위해 사용하는 과거 데이터의 길이, m은 변수의 개수를 나타낸다.
- 본 데이터셋에서는 모델이 과거 20개(2초)의 데이터를 사용하여 100시점(10초) 이후의 이상작동을 탐지할 수 있게 구성하였다.
- 학습을 위해 list type으로 편집된 데이터를 numpy array타입으로 변환한다.



⑤-4. LSTM-Autoencoder 임계값 지정을 위한 검증 데이터셋 구성

```
# LSTM-AE Threshold 지정을 위한 Validation 셋 구성

X_valid_normal, Y_valid_normal = X_test_normal[:880, :, :], Y_test_normal[:880]
X_test_normal, Y_test_normal = X_test_normal[880:, :, :], Y_test_normal[880:]
X_valid_anomal, Y_valid_anomal = X_test_anomal[:300, :, :], Y_test_anomal[:300]
X_test_anomal, Y_test_anomal = X_test_anomal[300:, :, :], Y_test_anomal[300:]

X_valid = np.vstack((X_valid_normal, X_valid_anomal))
Y_valid = np.hstack((Y_valid_normal, Y_valid_anomal))

X_test = np.vstack((X_test_normal, X_test_anomal))
Y_test = np.hstack((Y_test_normal, Y_test_anomal))
```

[코드 22] LSTM-Autoencoder 임계값 지정을 위한 검증 데이터셋 구성 코드

- 학습된 LSTM-Autoencoder가 새로운 데이터를 입력 받았을 때 정상/이상 여부를 판정하기 위한 기준인 임계값(Threshold)가 필요하다.
- 해당 임계값은 학습용 데이터에서 미리 구분한 검증용 데이터를 사용하여 정한다.
- 코드 22를 통해 LSTM-Autoencoder 임계값 지정을 위한 검증 데이터를 구분할 수 있다.
- 학습용 정상데이터 중 880개의 Sequence와 300개의 이상 데이터 Sequence를 검증 데이터로 사용한다.

⑤-5. 학습, 검증 및 평가용 데이터셋 구성확인

```
# 학습, 검증 및 평가용 데이터셋 구성확인

X_train, Y_train = np.array(X_train), np.array(Y_train)
X_valid, Y_valid = np.array(X_valid), np.array(Y_valid)
X_test, Y_test = np.array(X_test), np.array(Y_test)

print('X_train:', X_train.shape, 'Y_train:', Y_train.shape)
print('X_valid:', X_valid.shape, 'Y_valid:', Y_valid.shape)
print('X_test:', X_test.shape, 'Y_test:', Y_test.shape)
```

```
X_train: (14880, 20, 3) Y_train: (14880,)
X_valid: (1180, 20, 3) Y_valid: (1180,)
X_test: (4180, 20, 3) Y_test: (4180,)
```

[코드 23] 학습, 검증 및 평가용 데이터 구성확인 코드

- 코드 23을 통해 학습, 검증 및 평가용 데이터를 확인한다.



⑤-6. 모델 학습과정에서 사용할 검증 데이터 분리

검증데이터 중 정상 데이터만 분리

```
X_valid_0 = X_valid[Y_valid!=0]
X_valid_0.shape
```

```
(880, 20, 3)
```

[코드 24] 모델 학습과정에서 사용할 검증데이터 분리

- 코드 22를 통해 구성된 검증 데이터셋은 임곗값 지정을 위해 정상 데이터와 이상 데이터 모두가 포함되어있어, 정상 데이터만 사용하는 LSTM-Autoencoder의 학습 과정에서 사용이 불가하다.
- 코드 24를 통해 검증 데이터셋에서 정상 데이터만 추출하여 학습과정에서 사용한다.

[단계 ⑥] 모델 정의 및 생성

⑥-1. LSTM-Autoencoder 모델 정의 및 생성

LSTM-Autoencoder 모델 정의 및 생성

```
def LSTM_AE(sequence,n_features):
    lstm_ae = models.Sequential()
    # Encoder
    lstm_ae.add(layers.LSTM(64, input_shape=(sequence, n_features), return_sequences=True))
    lstm_ae.add(layers.LSTM(32, return_sequences=False))
    lstm_ae.add(layers.RepeatVector(sequence))
    # Decoder
    lstm_ae.add(layers.LSTM(32, return_sequences=True))
    lstm_ae.add(layers.LSTM(64, return_sequences=True))
    lstm_ae.add(layers.TimeDistributed(layers.Dense(n_features)))
    return lstm_ae
```

```
lstm_ae = LSTM_AE(20, 3)
lstm_ae.summary()
```

```
Model: "sequential_1"
Layer (type)                 Output Shape              Param #
-----
lstm_4 (LSTM)                 (None, 20, 64)           17408
lstm_5 (LSTM)                 (None, 32)               12416
repeat_vector_1 (RepeatVect  (None, 20, 32)           0
or)
lstm_6 (LSTM)                 (None, 20, 32)           8320
lstm_7 (LSTM)                 (None, 20, 64)           24832
time_distributed_1 (TimeDis  (None, 20, 3)            195
tributed)
-----
Total params: 63,171
Trainable params: 63,171
Non-trainable params: 0
```

[코드 25] 모델 생성 코드



- 코드 25를 통해 LSTM-Autoencoder 모델을 정의하고 생성한다.
- 본 LSTM-Autoencoder 모델은 각각 2개의 LSTM 레이어로 이루어진 Encoder와 Decoder로 구성되어있으며, Sequence의 길이는 20, 변수의 개수는 3개이다.

[단계 ⑦] 모델 학습 및 평가

⑦-1. 모델 학습

#모델 학습 과정에서의 Callbacks 선언

```
reduce_lr = ReduceLROnPlateau(monitor='val_loss', factor=0.7, patience=50, verbose=1)
es = EarlyStopping(monitor='val_loss', min_delta=0.00001, patience=120, verbose=1, mode='min',
restore_best_weights=True)
```

#모델 컴파일 및 학습 진행

```
lstm_ae.compile(loss='mse', optimizer=optimizers.Adam(0.001))
```

fit

```
history = lstm_ae.fit(X_train, X_train,
epochs=800, batch_size=128,
callbacks=[reduce_lr, es], validation_data=(X_valid_0, X_valid_0))
```

```
Epoch 787/800
117/117 [=====] - 1s 6ms/step - loss: 0.0022 - val_loss: 0.0027 - lr: 8.2354e-05
Epoch 788/800
117/117 [=====] - 1s 7ms/step - loss: 0.0022 - val_loss: 0.0027 - lr: 8.2354e-05
Epoch 789/800
117/117 [=====] - 1s 7ms/step - loss: 0.0022 - val_loss: 0.0027 - lr: 8.2354e-05
Epoch 790/800
117/117 [=====] - 1s 6ms/step - loss: 0.0022 - val_loss: 0.0027 - lr: 8.2354e-05
Epoch 791/800
117/117 [=====] - 1s 7ms/step - loss: 0.0022 - val_loss: 0.0027 - lr: 8.2354e-05
Epoch 792/800
117/117 [=====] - 1s 7ms/step - loss: 0.0022 - val_loss: 0.0027 - lr: 8.2354e-05
Epoch 793/800
117/117 [=====] - 1s 6ms/step - loss: 0.0022 - val_loss: 0.0027 - lr: 8.2354e-05
Epoch 794/800
117/117 [=====] - 1s 7ms/step - loss: 0.0022 - val_loss: 0.0027 - lr: 8.2354e-05
Epoch 795/800
117/117 [=====] - 1s 6ms/step - loss: 0.0022 - val_loss: 0.0027 - lr: 8.2354e-05
Epoch 796/800
117/117 [=====] - 1s 7ms/step - loss: 0.0022 - val_loss: 0.0027 - lr: 8.2354e-05
Epoch 797/800
117/117 [=====] - 1s 7ms/step - loss: 0.0022 - val_loss: 0.0027 - lr: 8.2354e-05
Epoch 798/800
117/117 [=====] - 1s 7ms/step - loss: 0.0022 - val_loss: 0.0027 - lr: 8.2354e-05
Epoch 799/800
117/117 [=====] - 1s 7ms/step - loss: 0.0022 - val_loss: 0.0027 - lr: 8.2354e-05
Epoch 800/800
117/117 [=====] - 1s 7ms/step - loss: 0.0022 - val_loss: 0.0027 - lr: 8.2354e-05
```

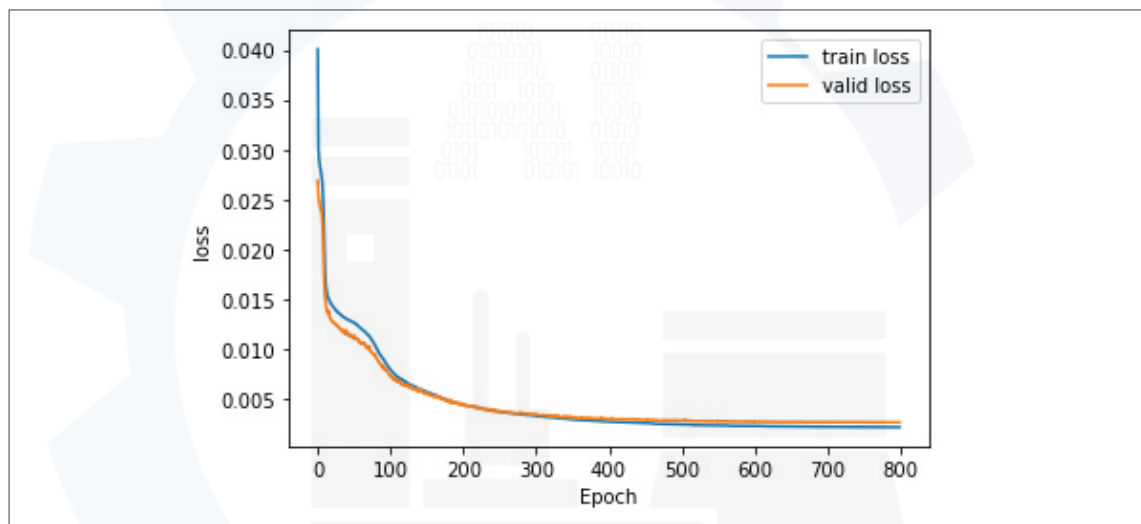
[코드 26] LSTM-Autoencoder 모델 학습 및 테스트

- 코드 26을 통해 LSTM-Autoencoder 모델의 학습을 진행한다.
- LSTM-Autoencoder는 입력받은 원본 데이터와 복원한 데이터의 오차를 바탕으로 학습을 진행하므로 Loss 함수는 MeanSquaredError를 사용하였다.

- 학습용 데이터에서 미리 분리한 검증용 정상 데이터를 사용하여 검증을 진행한다.
- 최적의 학습을 위해, 몇회 이상의 학습 Epoch이 진행 되었음에도 Loss의 감소가 이루어지지 않을 때 학습률을 감소시키는 학습률 반감(Learning Rate Decaying)을 적용하였다. 이는 모델이 최적의 가중치를 찾기 위해 더 세밀한 탐색 과정을 거치게 하는 효과가 존재한다.
- 또한, 모델의 Loss 함수가 수렴하였음에도 학습을 계속 진행할 때 학습을 종료하는 조기종료(Early Stopping)를 적용하였다. 이는 과적합을 방지하는 효과가 존재한다.

#모델 학습 반복수에 따른 loss 감소 시각화

```
plt.plot(history.history['loss'], label='train loss')
plt.plot(history.history['val_loss'], label='valid loss')
plt.legend()
plt.xlabel('Epoch'); plt.ylabel('loss')
plt.show()
```



[코드 27] 모델 학습 반복수에 따른 loss 감소 시각화 코드

- 코드 27을 통해 모델 학습 과정에서의 loss 감소를 시각화한다.
- 시각화 결과, 200~300 Epoch부터 학습 및 검증 데이터 loss가 잘 수렴되는 것을 확인할 수 있다.

⑦-2. LSTM-Autoencoder 임꺽값 설정 및 임꺽값 검증

#모델 출력값과 입력값의 재구성 오차를 계산하기위해 데이터 차원을 감소시키는 함수

```
def flatten(X):
    flattened = np.empty((X.shape[0], X.shape[2]))
    for i in range(X.shape[0]):
        flattened[i] = X[i, X.shape[1]-1, :]
    return(flattened)
```

[코드 28] 재구성 오차를 계산하기위해 데이터 차원을 감소시키는 함수 코드

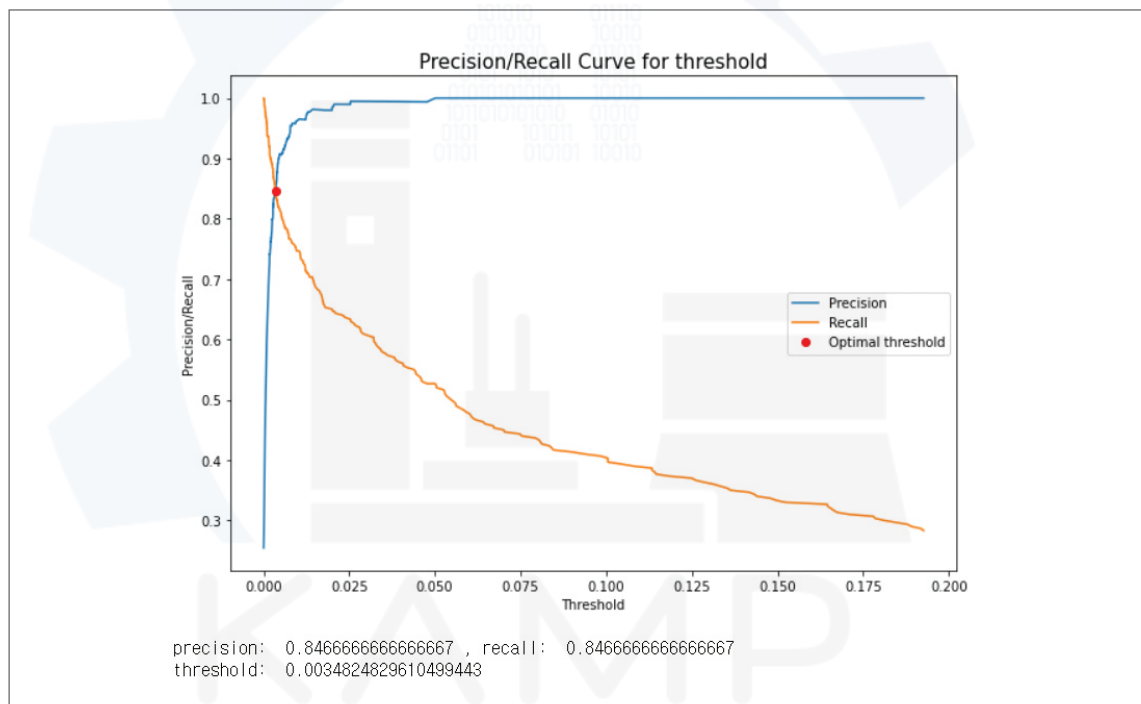
LSTM-Autoencoder 임계값 지정 및 임계값에 따른 정밀도와 재현율 곡선 시각화

```
valid_x_predictions = lstm_ae.predict(X_valid)
mse = np.mean(np.power(flatten(X_valid) - flatten(valid_x_predictions), 2), axis=1)

precision, recall, threshold = metrics.precision_recall_curve(list(Y_valid), mse)
index_cnt = [cnt for cnt, (p, r) in enumerate(zip(precision, recall)) if p==r][0]
threshold_final = threshold[index_cnt]

plt.figure(figsize=(10,7))
plt.title('Precision/Recall Curve for threshold', fontsize=15)
plt.plot(threshold[threshold <= 0.2], precision[1:][threshold <= 0.2], label='Precision')
plt.plot(threshold[threshold <= 0.2], recall[1:][threshold <= 0.2], label='Recall')
plt.plot(threshold_final, precision[index_cnt], 'o', color='r', label='Optimal threshold')
plt.xlabel('Threshold')
plt.ylabel('Precision/Recall')
plt.legend()
plt.show()

print('precision: ', precision[index_cnt], ', recall: ', recall[index_cnt])
print('threshold: ', threshold_final)
```

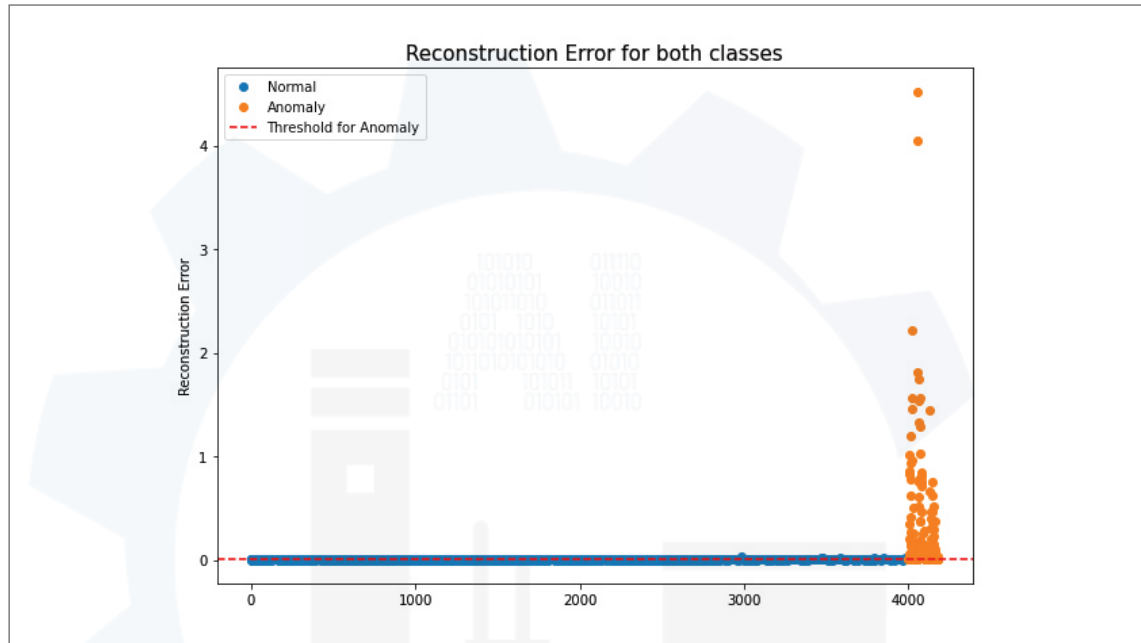


[코드 29] LSTM-Autoencoder 임계값 지정 및 임계값에 따른 정밀도와 재현율 곡선 시각화 코드

- 코드 29를 통해 검증용 데이터에서 LSTM-Autoencoder 임계값 설정과 그에 따른 정밀도-재현율 곡선을 시각화 한다.
- 정밀도-재현율 곡선이 교차하는 지점의 임계값을 최적의 값으로 판단한다. 이때 정밀도와 재현율은 약 0.847이고 임계값은 약 0.0035이다.
- 가이드북과 실제 실행결과가 소폭 다를 수 있으나, 이는 모델의 학습과정에서 초기 가중치가 매번 다르게 초기화되어 발생한 결과로서, 최종 학습 결과에 큰 영향을 주지 않는다.

설정한 임계값 검증

```
test_x_predictions = lstm_ae.predict(X_test)
mse = np.mean(np.power(flatten(X_test) - flatten(test_x_predictions), 2), axis=1)
plt.figure(figsize=(10,7))
plt.title('Reconstruction Error for both classes', fontsize=15)
plt.plot(np.where(Y_test==0)[0], mse[Y_test==0], marker='o', linestyle='', label='Normal')
plt.plot(np.where(Y_test==1)[0], mse[Y_test==1], marker='o', linestyle='', label='Anomaly')
plt.axhline(threshold_final, 0, len(Y_test), color='r', linestyle='--', label='Threshold for Anomaly')
plt.legend()
plt.ylabel('Reconstruction Error')
plt.show()
```



[코드 30] 설정된 임계값 검증 코드

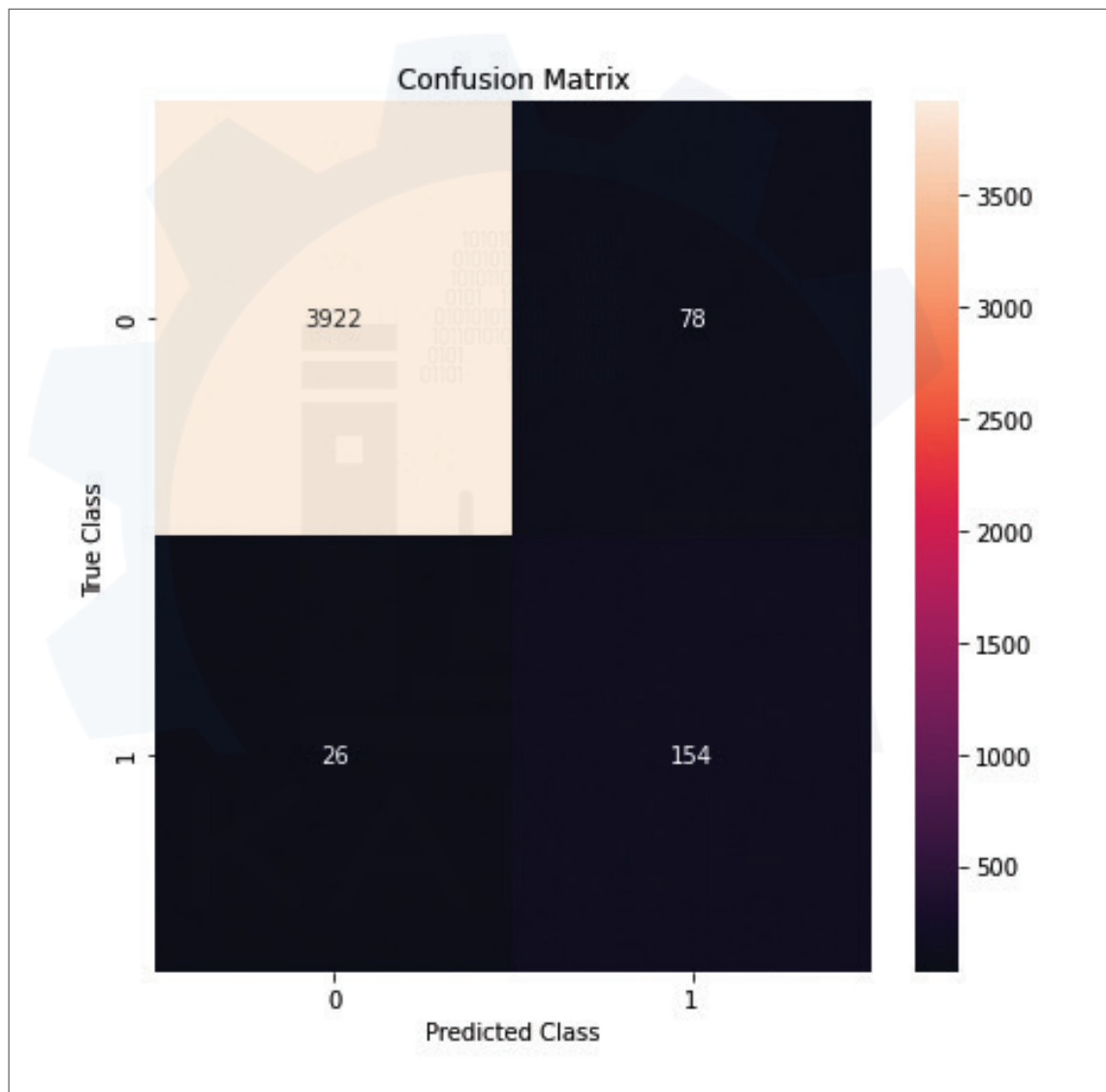
- 코드 30을 통해 설정된 임계값이 이상치 탐지에 적절한지 검증한다.
- 시각화를 통해 보았을 때, 설정한 임계값을 기준으로 정상 데이터와 이상 데이터의 구분이 잘 이루어짐을 확인할 수 있다.

⑦-3. 학습된 LSTM-Autoencoder 평가

혼동행렬 시각화

```
pred_y = [1 if e > threshold_final else 0 for e in mse]

conf_matrix = metrics.confusion_matrix(list(Y_test), pred_y)
plt.figure(figsize=(7, 7))
sns.heatmap(conf_matrix, xticklabels=[0,1], yticklabels=[0,1], annot=True, fmt='d')
plt.title('Confusion Matrix')
plt.xlabel('Predicted Class')
plt.ylabel('True Class')
plt.show()
```



[코드 31] LSTM-Autoencoder 평가 데이터 혼동행렬 시각화 코드

- 코드 31을 통해 LSTM-Autoencoder 모델의 혼동행렬을 시각화 할 수 있다.
- 해당 모델은 4000개의 평가용 정상 데이터 중, 3922개를 정확하게 분류하였고 78개를 오분류하였다.



- 180개의 이상 데이터에서는 154개를 정확하게 분류하였고 26개를 오분류하였다.

```
# LSTM-Autoencoder 정확도 출력  
print('Accuracy : {}'.format(metrics.accuracy_score(list(Y_test), pred_y)))
```

```
Accuracy : 0.9751196172248804
```

[코드 32] LSTM-Autoencoder 평가 데이터 정확도 출력 코드

```
# LSTM-Autoencoder 정확도 출력  
print('F1-Score : {}'.format(metrics.f1_score(list(Y_test), pred_y)))
```

```
F1-Score : 0.7475728155339806
```

[코드 33] LSTM-Autoencoder 평가 데이터 F1-Score 출력 코드

- 코드 32와 33를 통해서 학습된 LSTM-Autoencoder 모델을 평가한다.
- 코드 32를 통해 모델의 정확도를 출력할 수 있다.
- 코드 33를 통해 모델의 F1-Score를 출력할 수 있다.
- 해당 모델은 정확도 97.51%, F1-Score 74.76%를 기록하였다.



3. 유사 타현장의 「소성가공 예지보전 AI 데이터셋」 분석 적용

3.1 본 분석이 적용 가능한 제조 현장 소개

- 본 분석에서는 유압펌프 상부진동, 유압펌프 하부진동, 전류 데이터를 활용하여 소성가공 예지보전 모델 구축을 진행하였다. 시계열 이상치 탐지 알고리즘을 사용하여 모델을 개발하고, 모델은 시계열 데이터의 패턴을 학습하여 미래의 고장을 탐지한다. 이러한 분석 프로세스를 통해 숙련된 작업자 뿐만 아니라 비숙련된 작업자 또한 데이터 기반의 유압펌프 고장을 사전에 감지할 수 있게 된다.
- 본 분석의 결과물인 소성가공 예지보전 모델 개발 및 구축을 위해서는 설비 상태의 주요 요소들에 대한 데이터 수집 및 관리가 중요하다. 이를 위해 유압펌프 설비의 상태를 나타내는 상부 진동, 하부 진동, 전류 변수들을 수집하여 모델링을 진행하였다. 따라서, 유압펌프에 센서를 설치하여 위와 같은 변수를 수집한다면, 설비 예지보전이 필요한 다른 소성가공 프레스 공정 기업들이 손쉽게 활용할 수 있을 것으로 기대된다.
- 타 소성가공 세부 공정으로는 구부림, 디프 드로잉, 전단 등이 있다. 각 공정별로 작동원리가 조금씩 다르지만 공통적으로 유압펌프를 사용하여 제품 가공을 진행한다. 유압펌프 내부의 고장이 발생하는 경우, 진동 및 소음에 변화가 발생할 것이므로 유압펌프를 사용하여 제품을 가공하는 타 소성가공 공정에서도 본 분석에서 사용한 변수들을 수집하는 프로세스를 적용하여 예지보전 모델을 개발할 수 있을 것이다.
- 또한 소성가공이 아닌 사출 성형과 같은 타 공정에서도 유압펌프는 필수적이다. 본 분석의 프로세스를 원용하여 타 제조공정에서 유압펌프의 진동 및 전류 데이터를 수집하게 되면 예지보전 모델 구축이 가능할 것이다.

3.2 본 「소성가공 예지보전 AI 데이터셋」 분석을 원용하여 타 제조현장 적용시, 주요 고려사항

- 먼저 프레스 공정 설비의 유압펌프 데이터를 수집할 수 있는 시스템 구축이 선행되어야 한다. 본 분석에서 사용한 유압펌프 상부진동, 하부진동, 전류는 설비의 상태를 내포하고 있는 변수들로, 해당 변수들의 수집이 불가능한 경우에는 모델이 유압펌프의 상태를 학습할 수 없어 분석이 불가하다.
- 또한, 정상작동과 이상작동에 대한 정보가 필요하다. 본 분석에서는 정상작동 유압펌프 데이터를 바탕으로 LSTM-Autoencoder 알고리즘을 사용하여 시계열 이상탐지 모델을 학습시켰다. LSTM-Autoencoder는 입력받은 시계열 데이터와 출력 시계열 데이터 간의 재구성 오차를 바탕으로 이상치를 탐지한다. 모델이 정상작동 데이터를 입력받으면 작은 재구성 오차가 나오지만, 이상작동 데이터의 경우에는 모델이 학습하지 못한 패턴의 데이터이므로 재구성 오차가 매우 크게 발생한다. 만약 정상작동과 이상작동의 구분 정보가 존



재하지 않을 경우, 모델이 설비의 상태와 관계없이 학습을 진행하여 이상작동 데이터를 입력받았음에도 재구성 오차가 작게 출력되어 효과적인 예지보전이 불가하다.

- 유압펌프와 직접적인 연관성이 존재하나 본 분석에서 사용하지 않은 데이터로는 유압 데이터가 존재한다. 펌프 내부 기어에 고장이 발생하거나 마모가 발생하여 고장이 가까워지는 경우, 유압펌프의 출력 유압 데이터에 영향을 줄 것이다. 이러한 데이터를 추가적으로 수집하여 본 분석 프로세스에 적용할 경우, 모델의 성능향상을 기대할 수 있다.



3 부록_분석환경 구축을 위한 설치 가이드

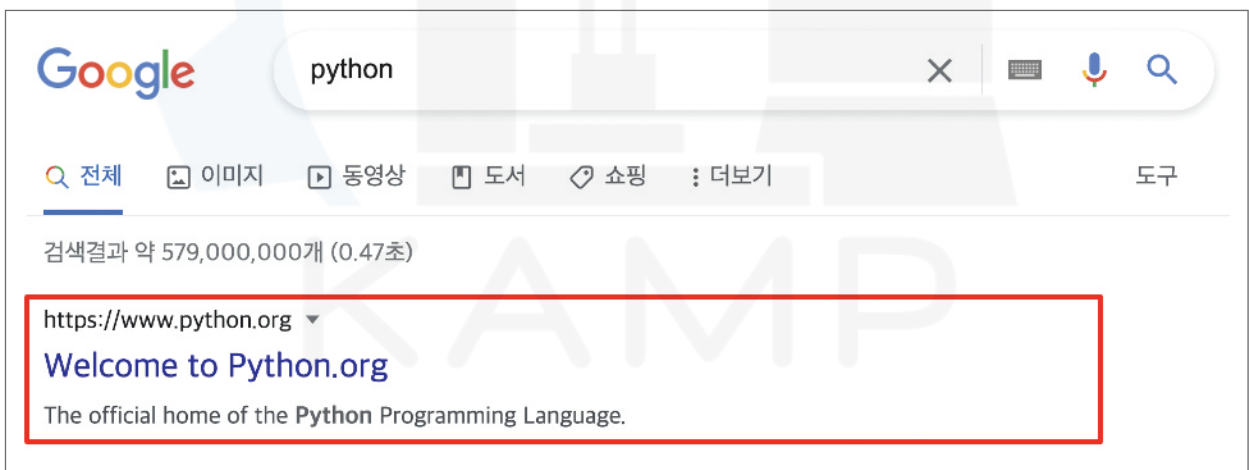
1. 파이썬(Python) 설치

- 파이썬은 컴퓨터 프로그래밍 언어로서, 데이터 분석 및 AI 분석모델 개발 등에 널리 사용되는 도구이다. 다운로드 및 설치가 간편하고 활용도가 높은 파이썬을 로컬 컴퓨터에 설치하는 방법을 안내한다.

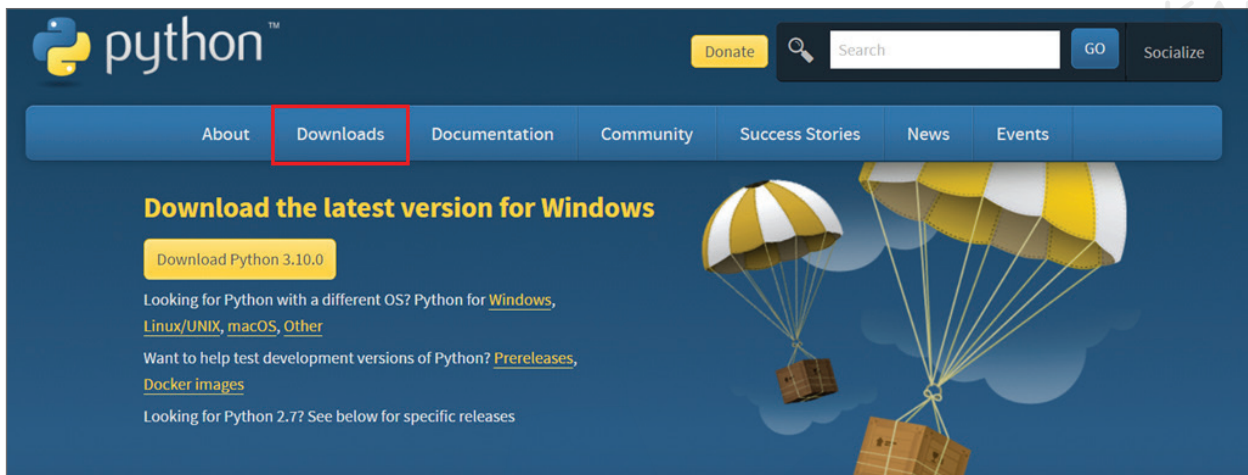
① google.com 등의 검색 엔진에 'python'을 검색한다.



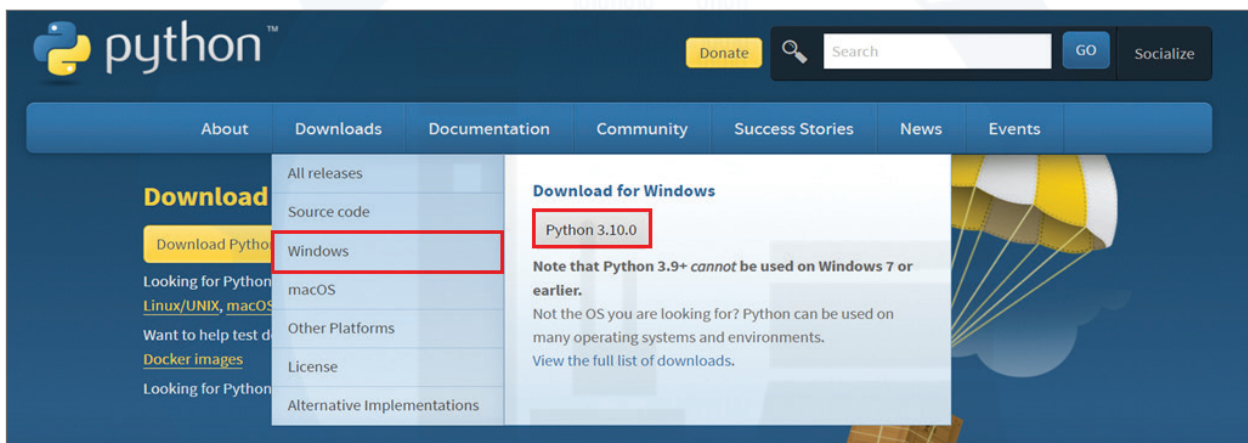
② 제일 처음에 보이는 'Welcome to Python.org'를 클릭한다.



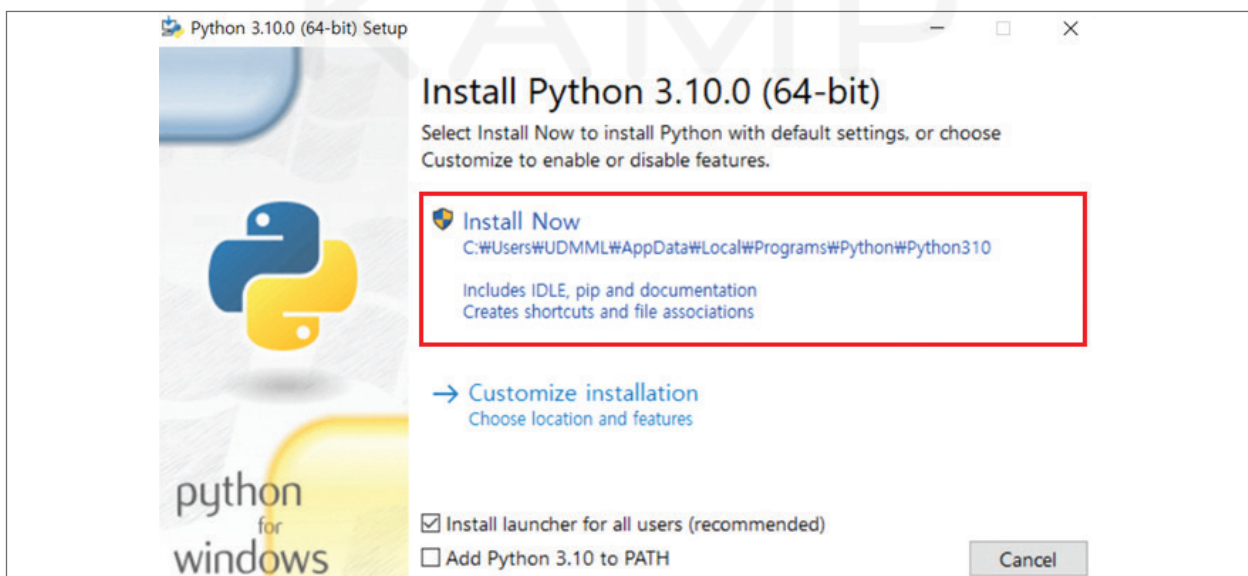
③ 클릭 후 보이는 페이지의 정면에서 상단 좌측 2번째 'Downloads'를 클릭한다.



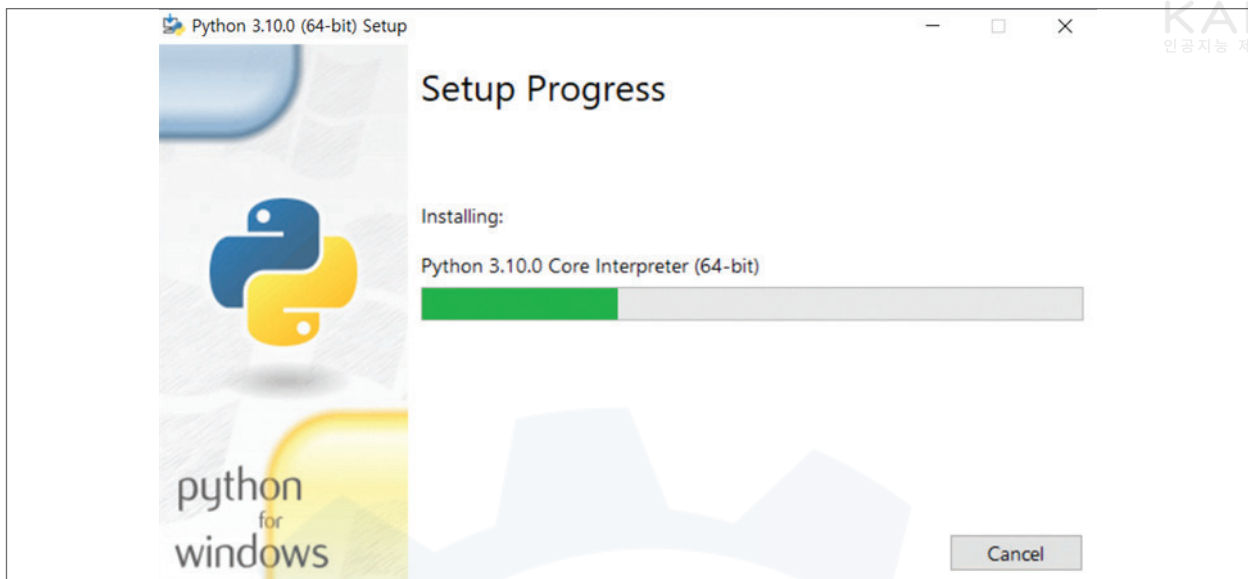
④ 컴퓨터의 운영체제에 따라 상단에서 세번째(macOS의 경우 네번째) 탭을 선택한 후, Python 3.10.0을 다운로드한다. (Python 버전은 업데이트 될 수 있다.)



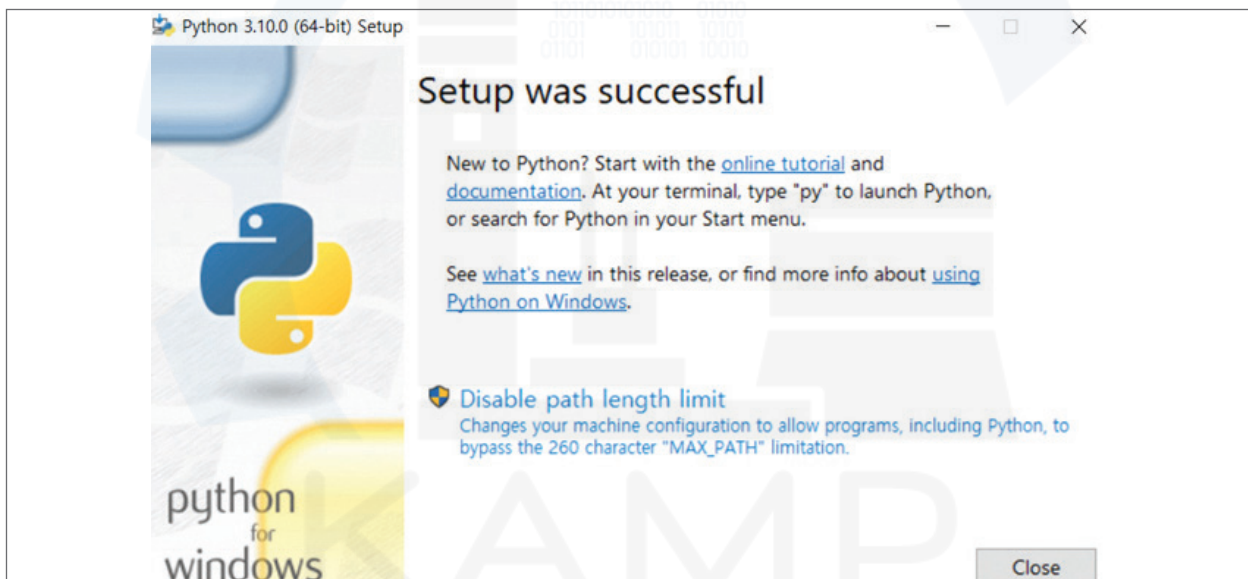
⑤ 아래와 같은 설치창이 뜨면, 'Install Now'를 클릭한다.



⑥ 아래와 같은 설치 진행창이 완료가 될 때까지 유지한다.



⑦ 아래와 같은 창이 나타나면 설치 완료된 것으로, 확인 후 종료한다. [설치완료]

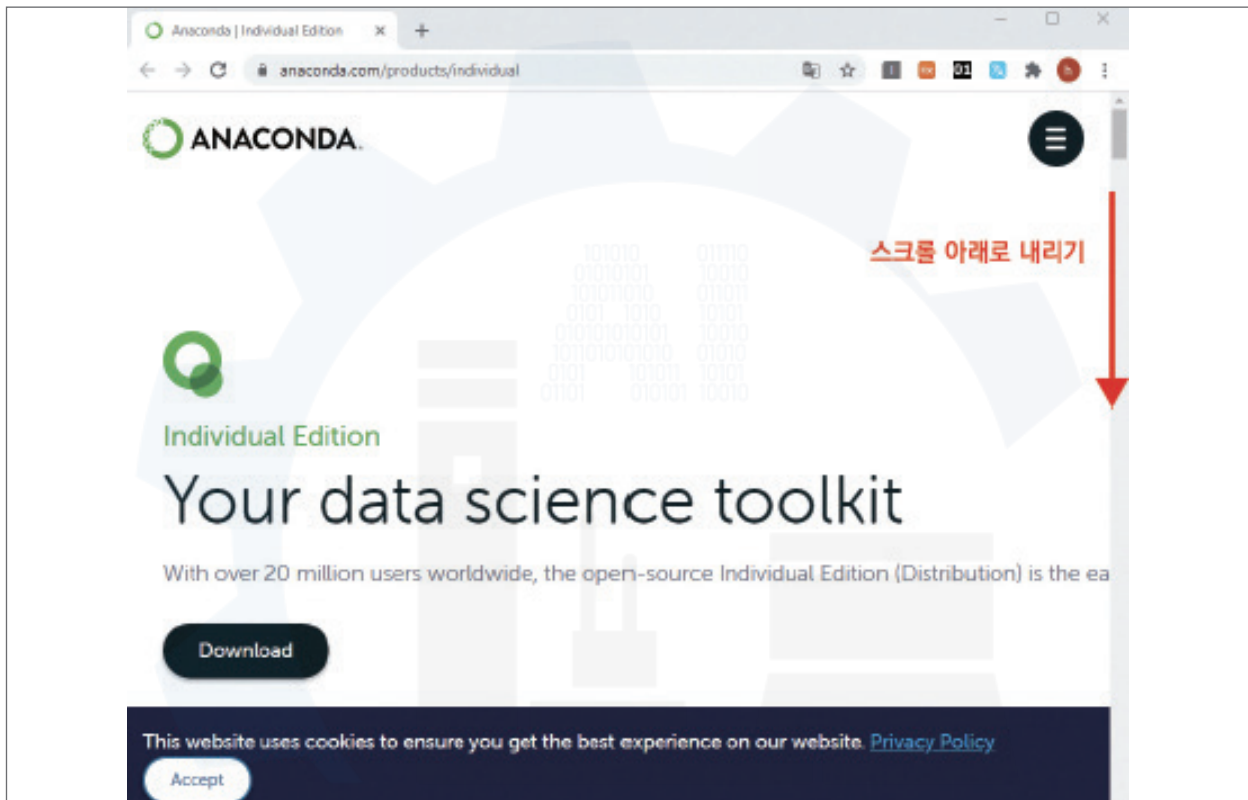




2. 아나콘다(Anaconda) 설치

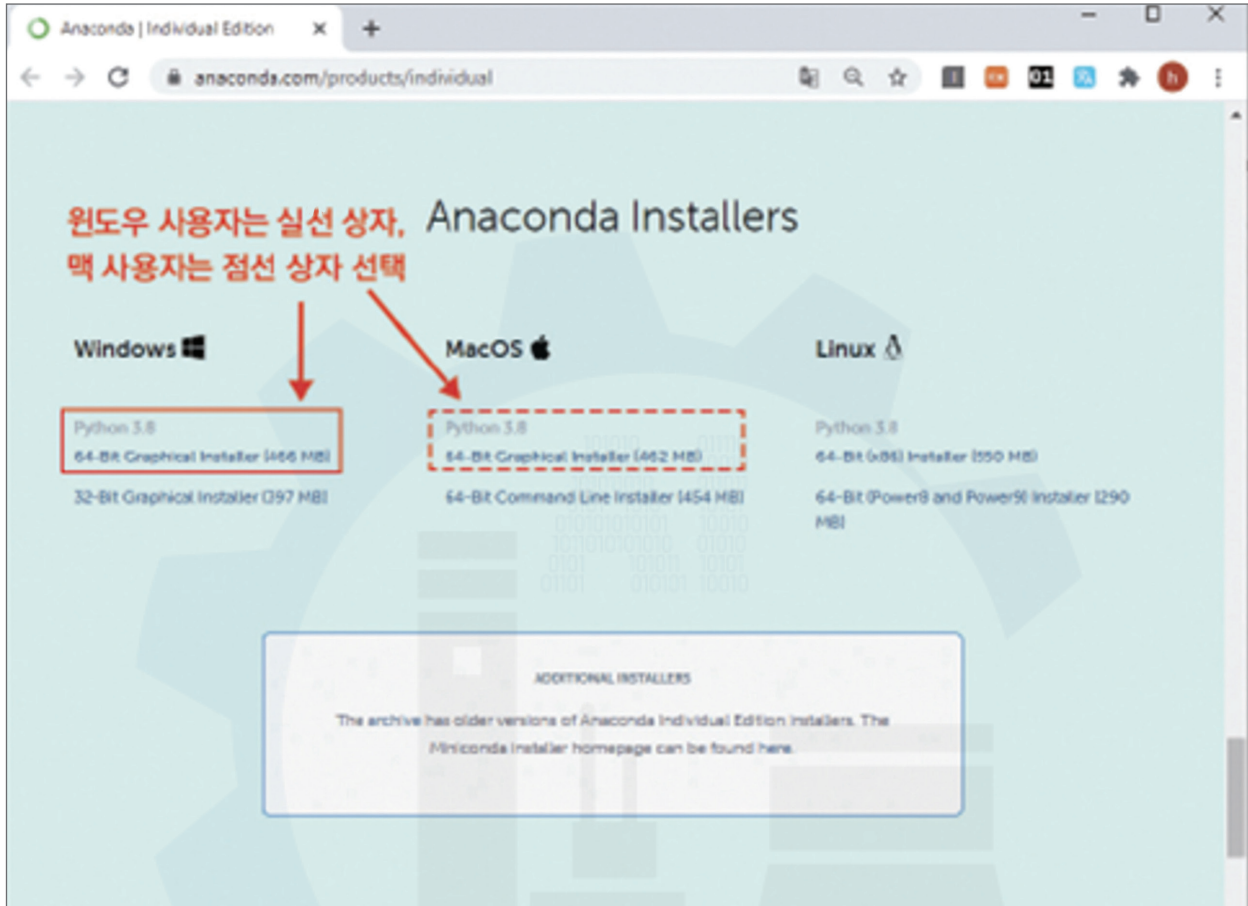
- 아나콘다란 파이썬 등의 데이터 분석 도구를 사용할 때 필요한 고급 기능 및 분석을 보조하는 도구이다. 아나콘다를 설치함으로써 다양한 기능들을 효율적으로 사용할 수 있으며, 결과물을 쉽게 확인할 수 있다. 아나콘다를 설치하고, 데이터를 분석할 수 있는 환경을 구축하는 방법에 대해 안내한다.

① <https://www.anaconda.com/distribution/> 로 접속한다.



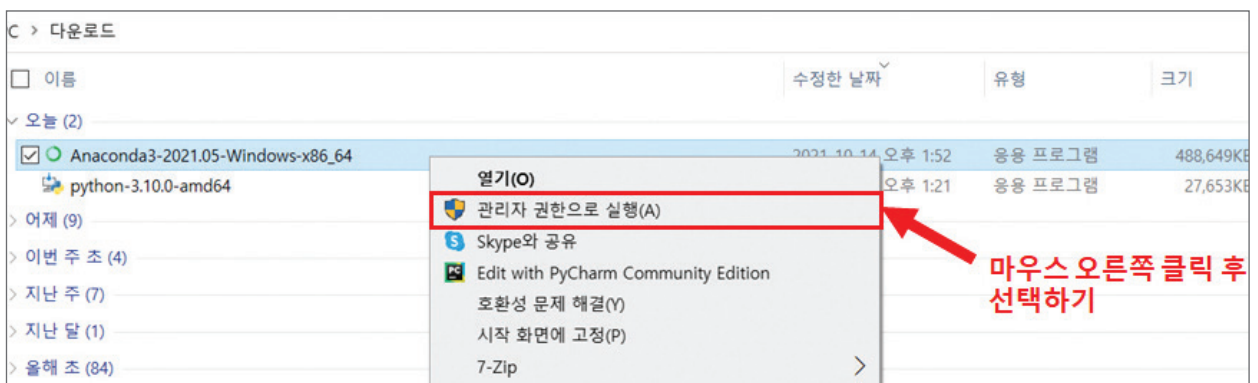
② 스크롤을 내려 다음의 화면을 확인한 후, 로컬 컴퓨터 사용 환경에 맞는 아나콘다 설치 파일을 다운로드한다.

- ▶ Windows : 64-Bit Graphical Installer
- ▶ MacOS : 64-Bit Graphical Installer

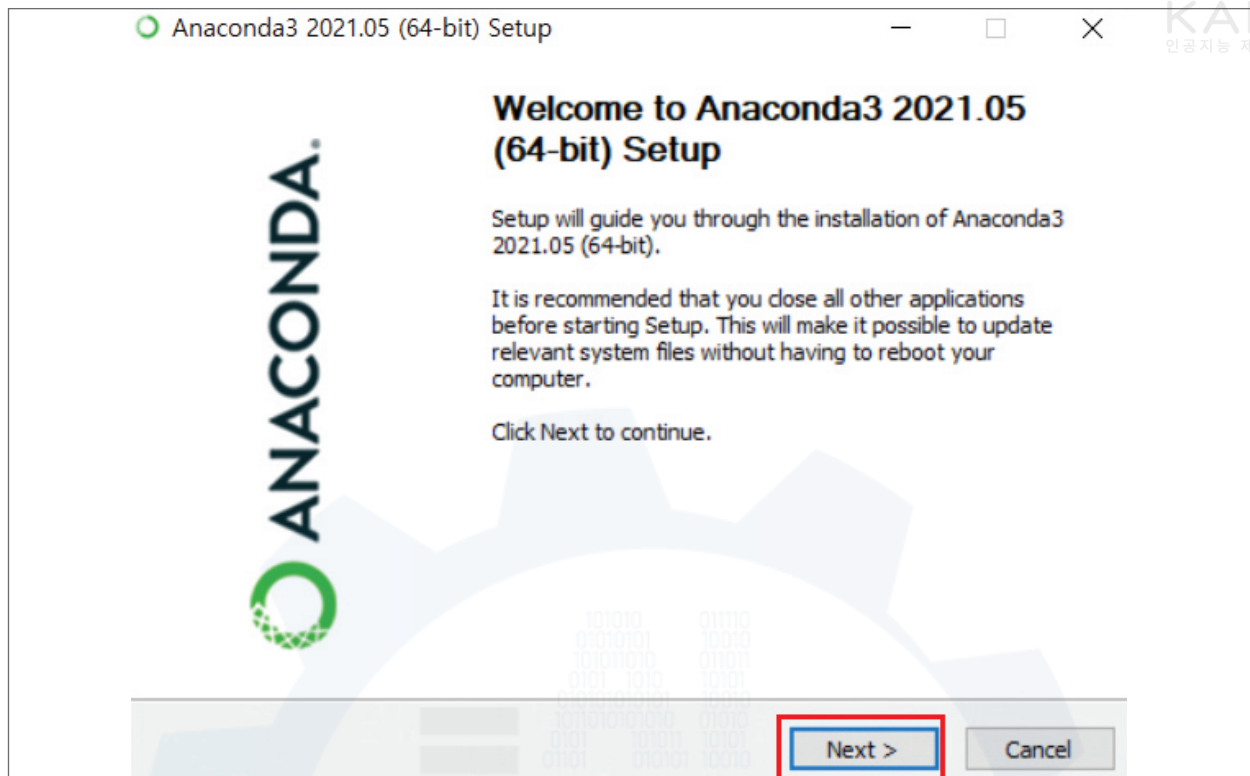


③ 다운로드한 아나콘다 설치 파일에서 마우스 오른쪽을 클릭한 후, 방패모양의 '관리자 권한으로 실행'을 클릭한다.

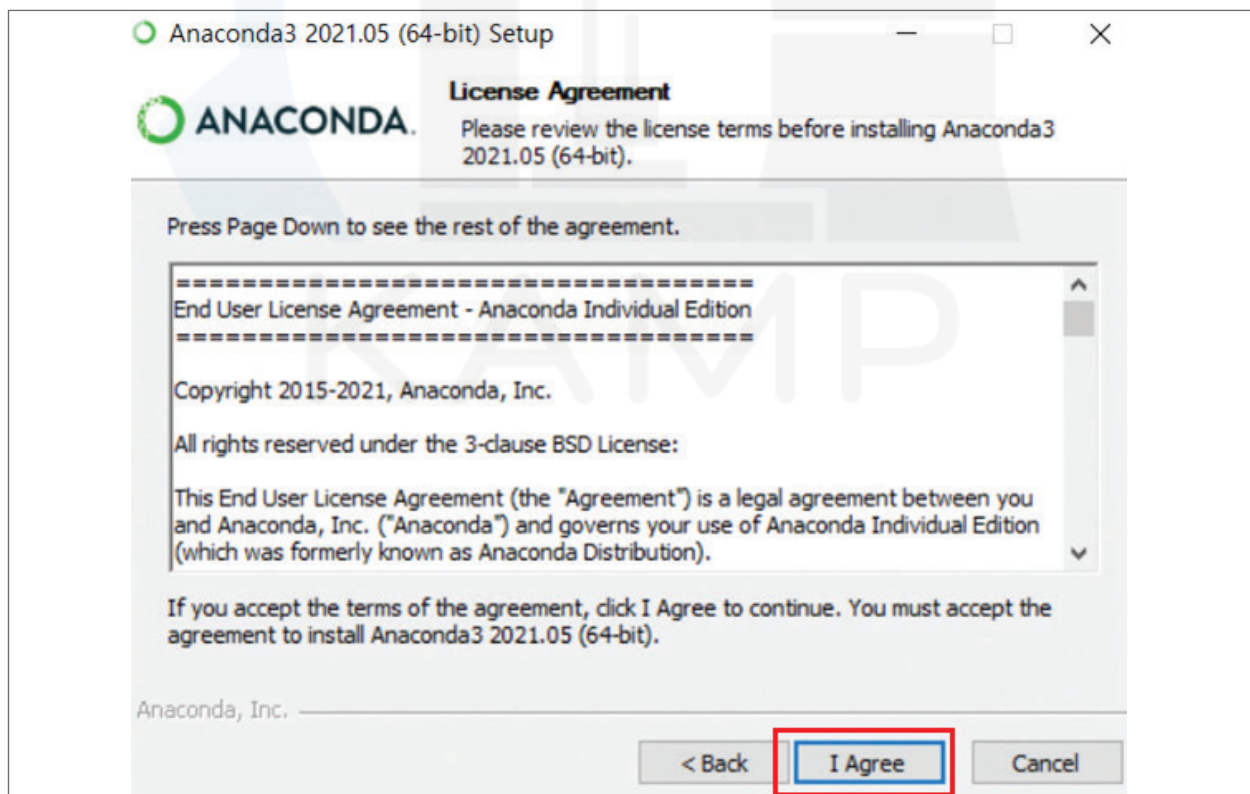
- ▶ (예) '다운로드' 폴더로 아나콘다를 다운로드한 경우



④ 설치 파일을 실행한 다음, 'Next' 버튼을 클릭한다.

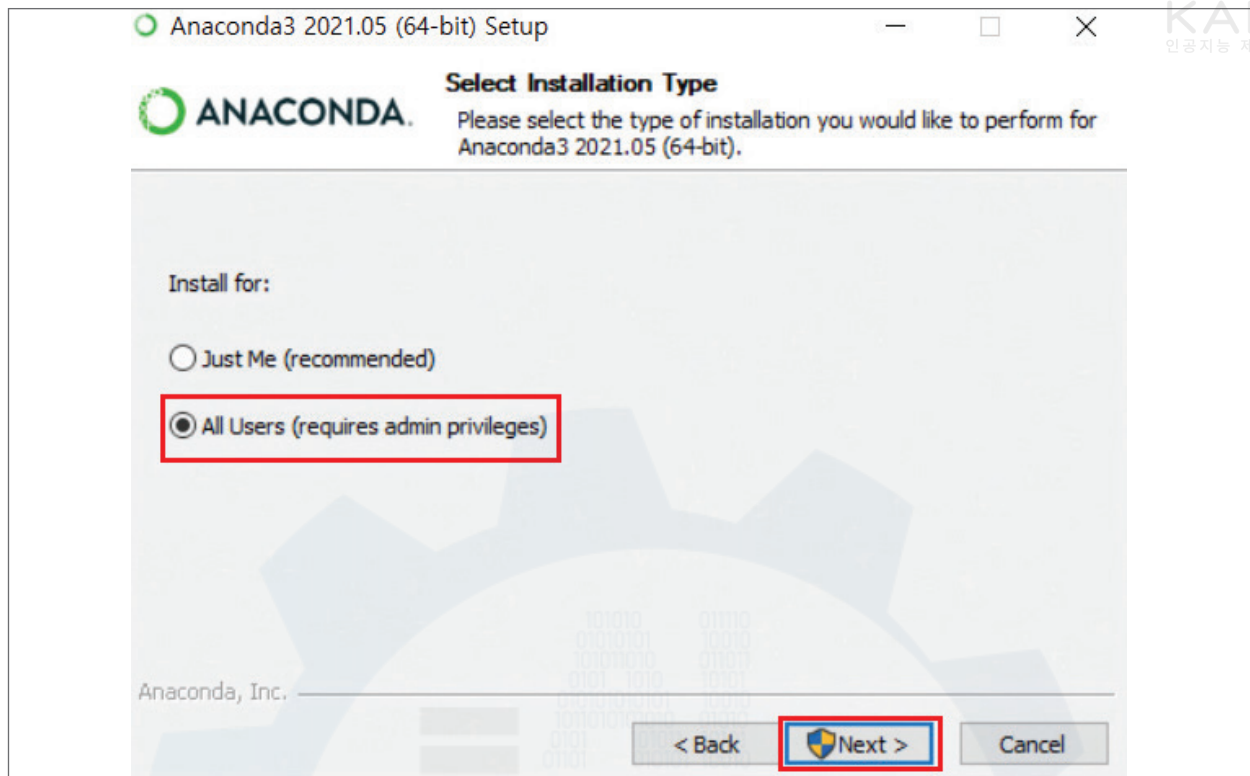


⑤ 다음 단계에서 아래와 같은 창이 나타나면, 'I Agree'를 선택한다.

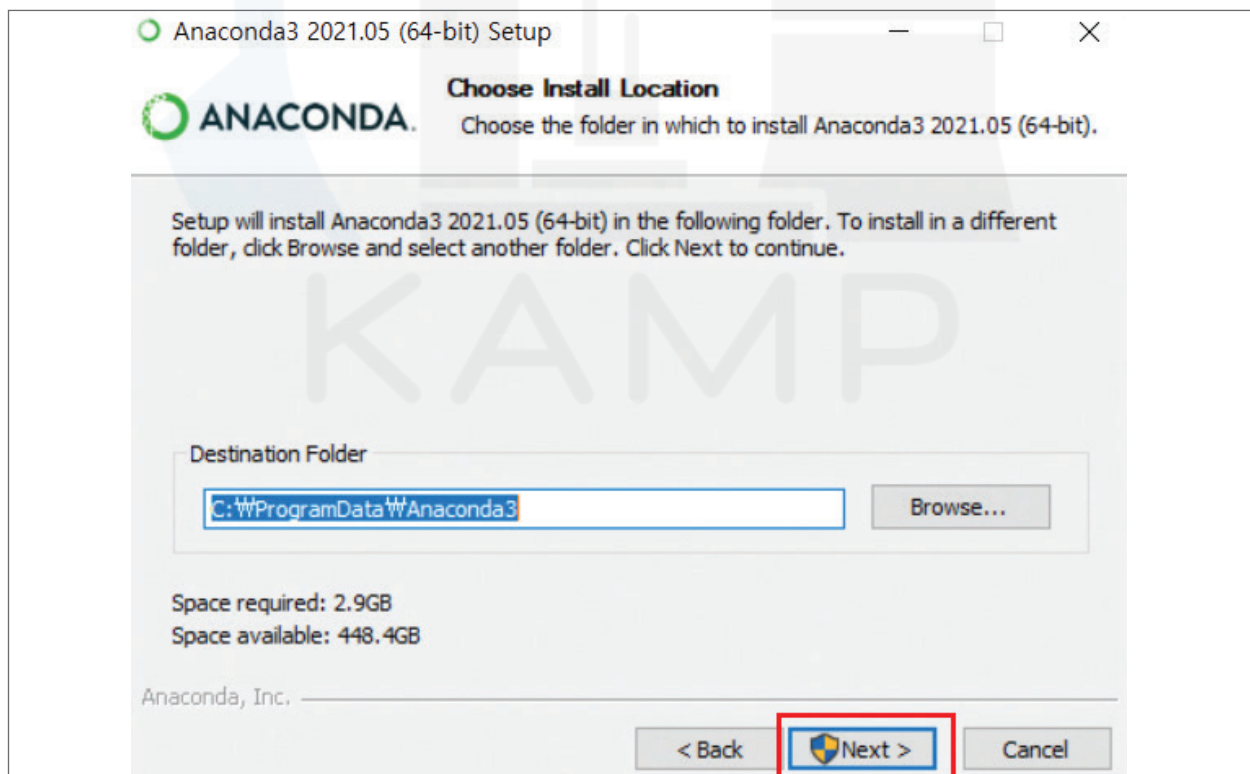




⑥ 아래와 같은 창이 나타나면, 'All Users'를 선택하여 다음 단계로 진행한다.

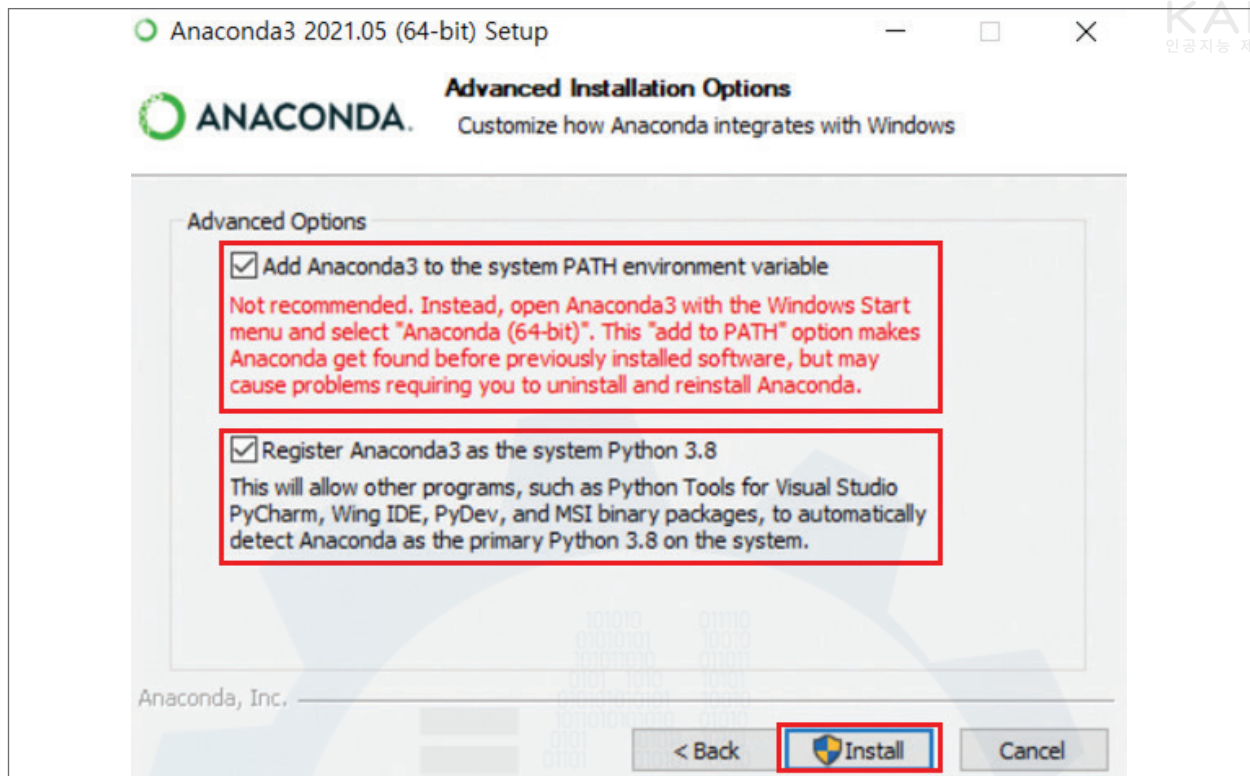


⑦ 다운로드할 경로를 설정한 뒤, 아래의 'Next'를 클릭한다.

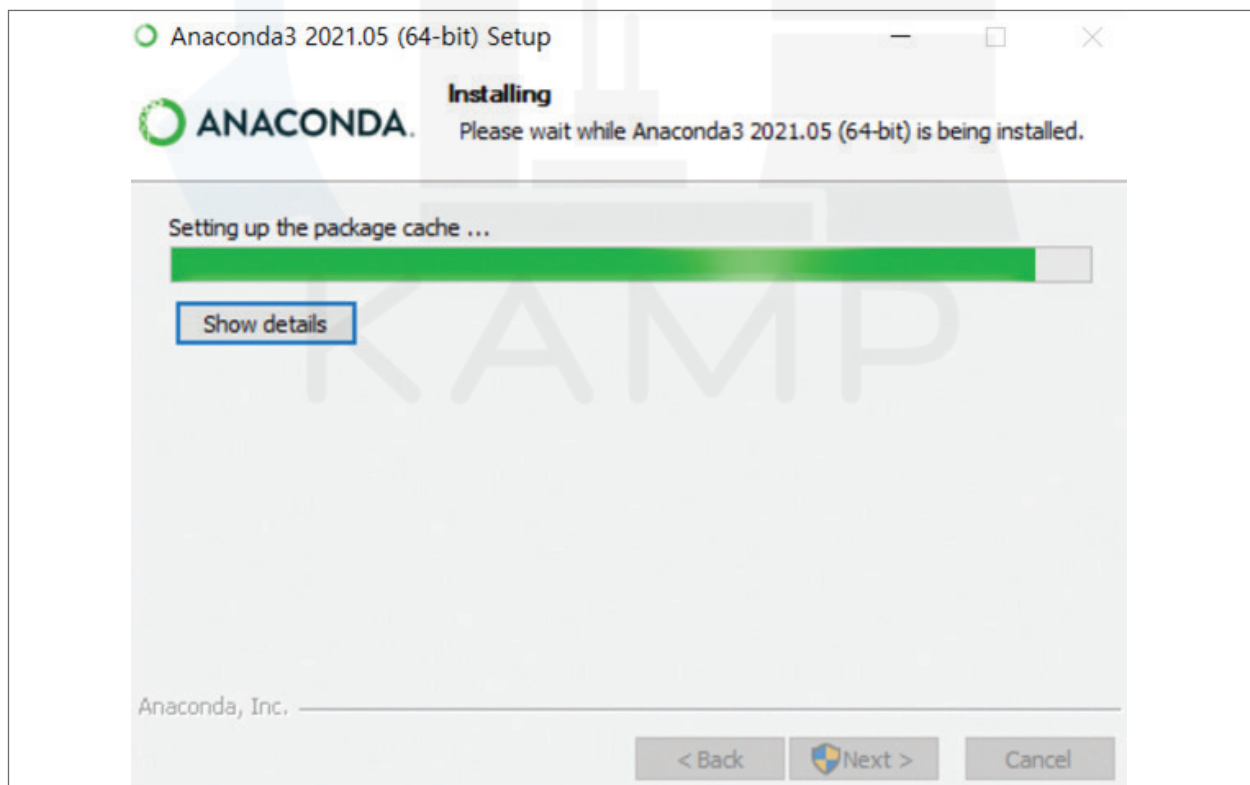




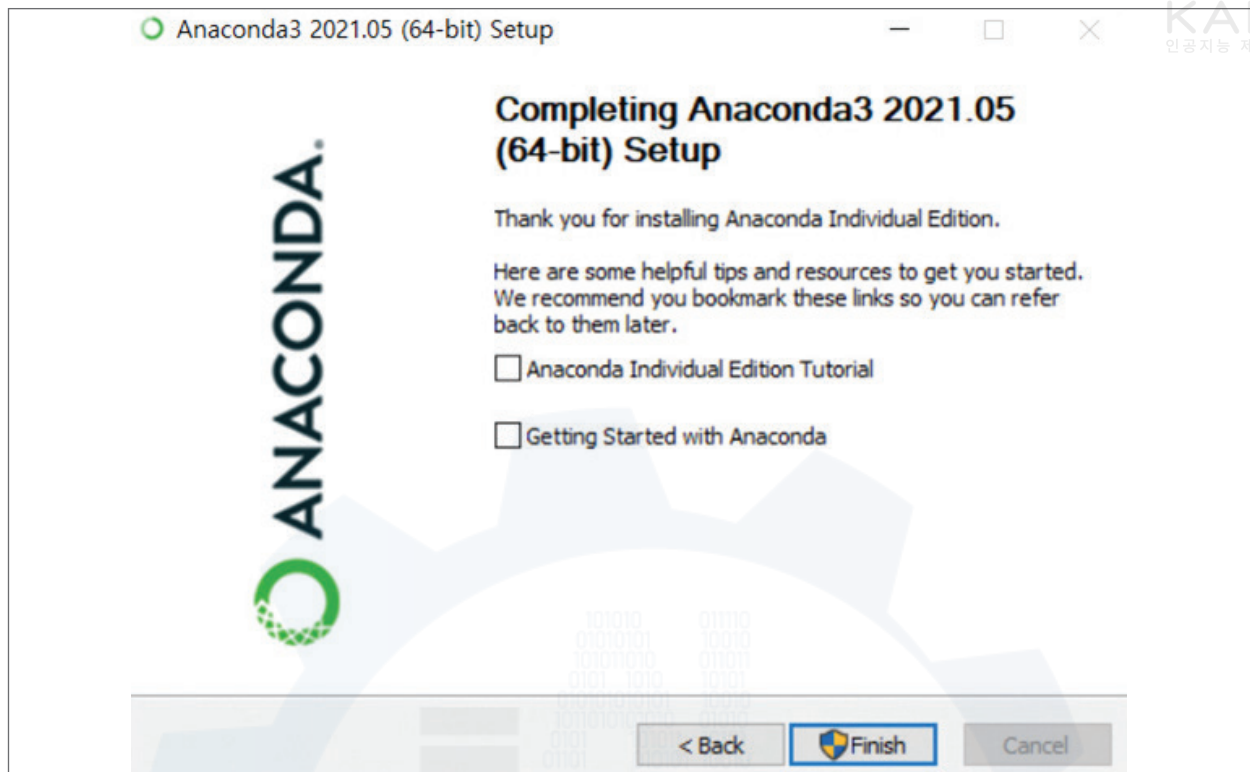
⑧ 고급 옵션 선택창에서 아래와 같이 모두 선택 후, 'Install'을 클릭한다.



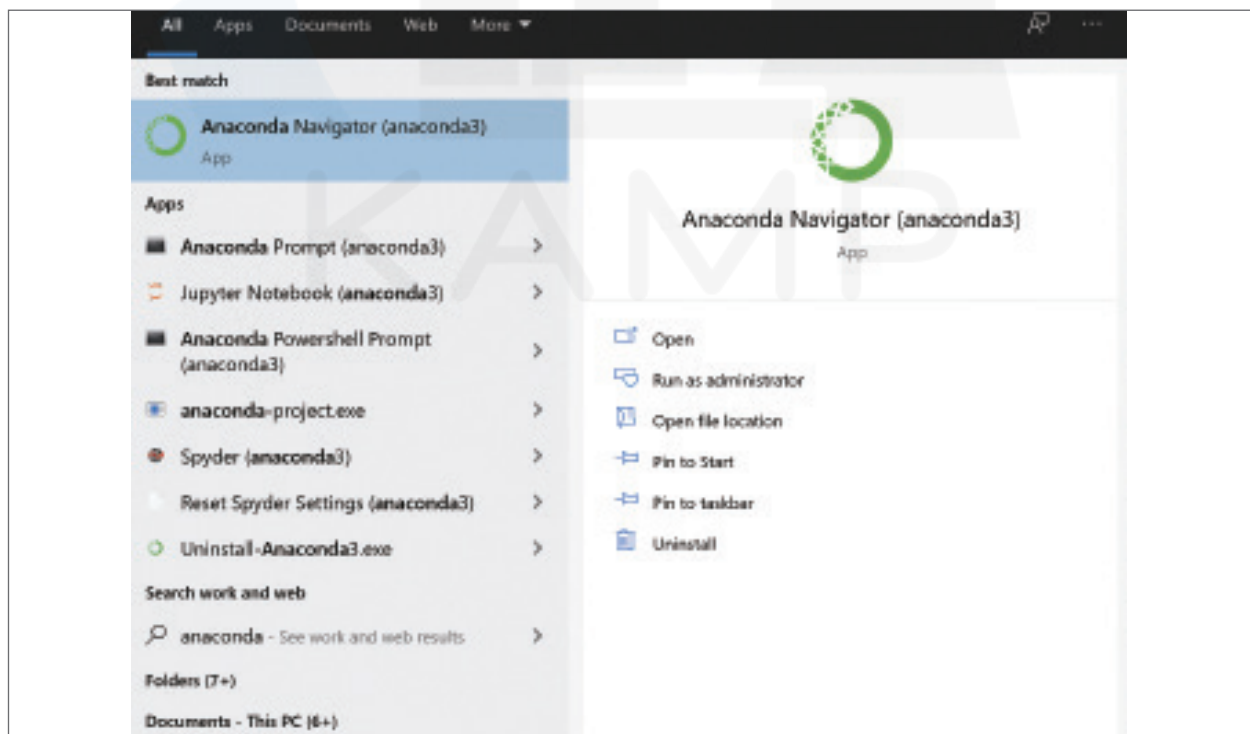
⑨ 다음과 같은 설치창으로 넘어가면, 완료가 될 때까지 대기한다. (5분이상 소요)



⑩ 마지막 화면에서 모두 체크 해제한 후, 'Finish'를 눌러 설치를 완료한다.



⑪ PC의 '시작 버튼(Windows logo)'을 눌러서 화면과 같이 anaconda prompt가 설치되었는지 확인한다. Anaconda Navigator, Anaconda Prompt, Jupyter Notebook 등의 다른 응용 프로그램들도 함께 확인이 된다면 설치가 완료된 것이다.

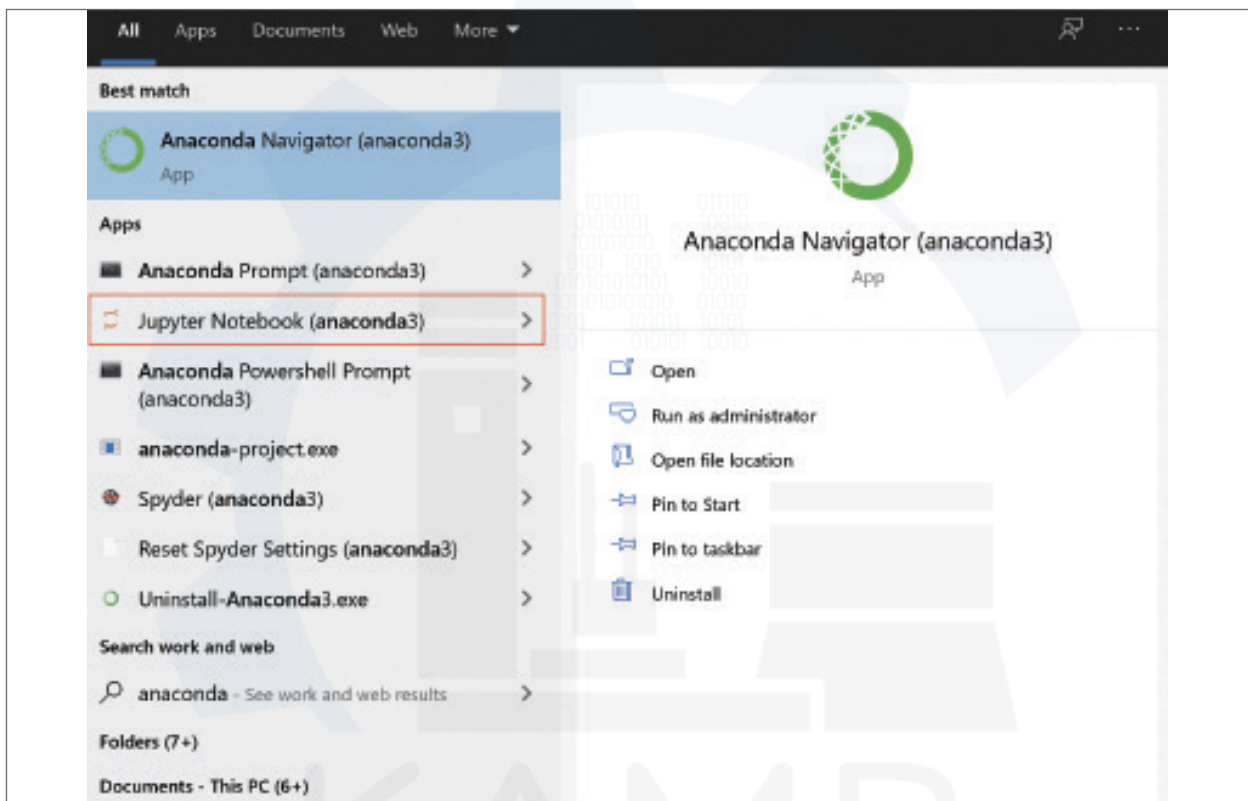




3. 주피터 노트북 (Jupyter Notebook) 실행

- 주피터 노트북은 사용자가 코딩(분석 코드 작성)을 할 수 있는 도구이다. 쉽게 비교하자면, 문서 도구로 마이크로소프트 사의 'Word' 프로그램이나, 한컴소프트 사의 '한글' 프로그램 등과 같은 도구라고 생각할 수 있다. 데이터 분석에 다양한 입력, 실행 도구가 있지만, 본 가이드북에서는 주피터 노트북을 활용하는 방법을 안내한다.

- ① PC의 '시작 버튼(Windows logo)'을 눌러서 화면과 같이 'anaconda'를 검색한 후 폴더 리스트 중 'Jupyter Notebook (anaconda3)'을 실행한다.





② Jupyter Notebook을 실행하면, 아래와 같이 2개의 윈도우가 실행된다.

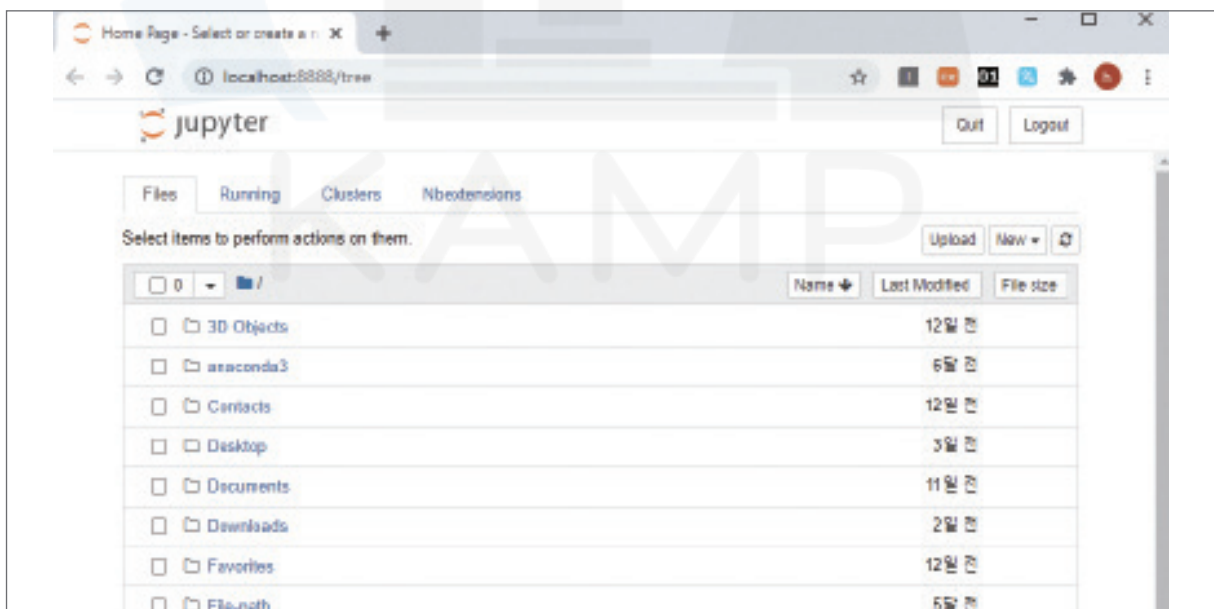
- (1) 아래 검은색 배경의 화면은 주피터 노트북이 실행되는 환경 상태를 나타내주는 상태 표시 창이다. 주피터 노트북을 사용하는 동안에는 종료하면 안된다.

```

Jupyter Notebook (anaconda3)
[I 12:39:38.566 NotebookApp] JupyterLab extension loaded from C:\Users\Wleedaeyoung\anaconda3\lib\site-packages\jupyterlab
[I 12:39:38.566 NotebookApp] JupyterLab application directory is C:\Users\Wleedaeyoung\anaconda3\share\jupyter\lab
[I 12:39:38.627 NotebookApp] Serving notebooks from local directory: C:\Users\Wleedaeyoung
[I 12:39:38.628 NotebookApp] The Jupyter Notebook is running at:
[I 12:39:38.628 NotebookApp] http://localhost:8888/?token=e6a3b83d88decc6e5b457c8fd8b7b86d823ba8bc31a88c2
[I 12:39:38.628 NotebookApp] or http://127.0.0.1:8888/?token=e6a3b83d88decc6e5b457c8fd8b7b86d823ba8bc31a88c2
[I 12:39:38.628 NotebookApp] Use Control-C to stop this server and shut down all kernels (twice to skip confirmation).
[C 12:39:39.053 NotebookApp]

To access the notebook, open this file in a browser:
file:///C:/Users/leedaeyoung/AppData/Roaming/jupyter/runtime/nbserver-6568-open.html
Or copy and paste one of these URLs:
http://localhost:8888/?token=e6a3b83d88decc6e5b457c8fd8b7b86d823ba8bc31a88c2
or http://127.0.0.1:8888/?token=e6a3b83d88decc6e5b457c8fd8b7b86d823ba8bc31a88c2
    
```

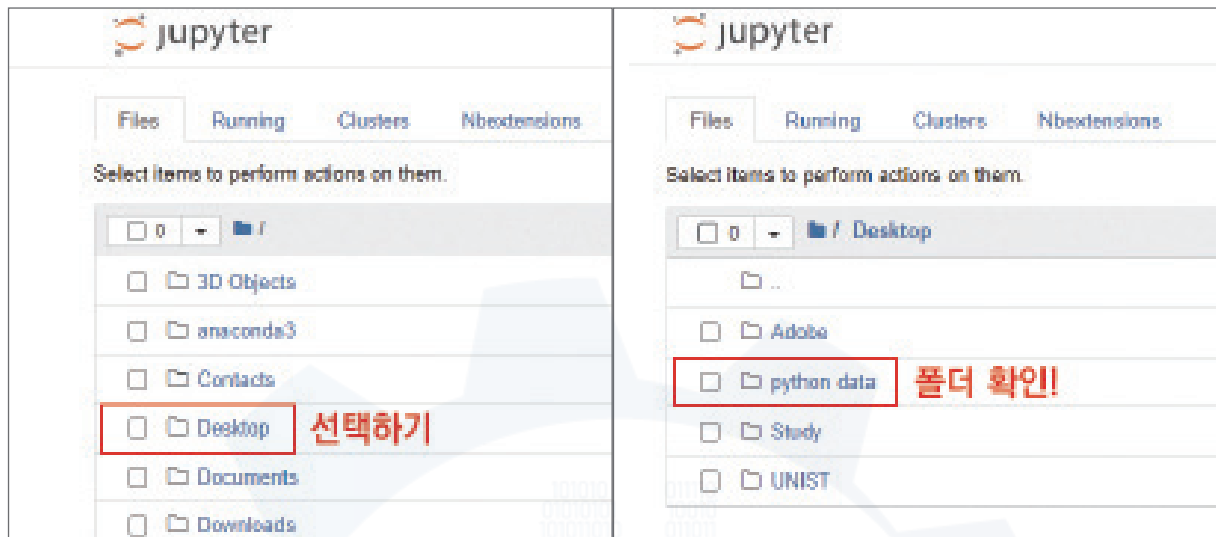
- (2) 기본 브라우저(크롬, 엣지 등)에서 주피터 노트북이 열린다. (다른 확장 프로그램 사용하고 싶다면 기본 브라우저를 변경해주어야 한다.)



③ 데이터 분석을 진행할 폴더를 생성한다.

▶ (예) '바탕화면'에 'python data' 폴더를 생성한 다음, 파이썬 코드 실행하고 저장한다.

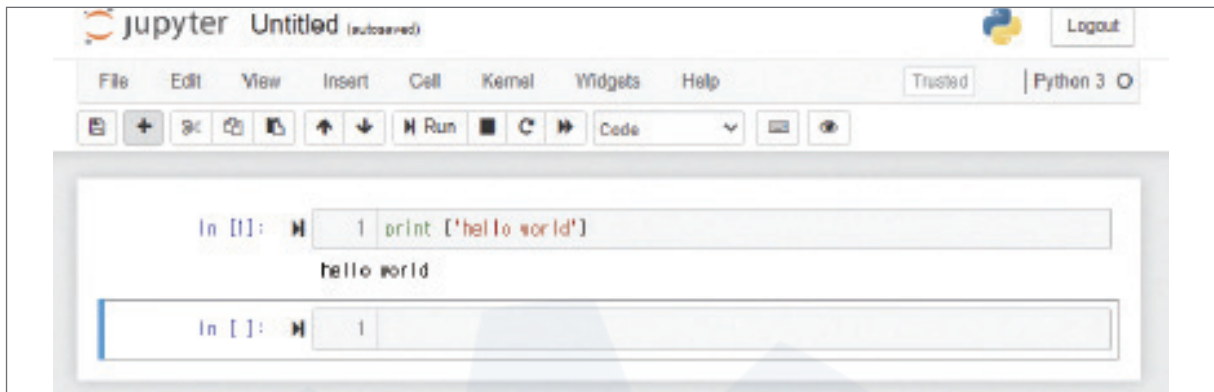
(1) 주피터 노트북에서 'python data' 폴더를 확인한다.



(2) python data에서 파이썬 파일을 생성한다. 오른쪽 상단의 'New'를 누른 후, 'Python 3'을 선택한다.



- (3) 'hello world' 출력을 확인해본다. 보이는 In [] 우측 회색 창에 print('hello world')를 입력한 다음, 상단의  Run 버튼 혹은 shift + Enter 키를 눌러서 실행한다. 코드 파일의 확장자는 '*.ipynb'로 저장된다.





4 부록_데이터 품질 전처리 [실습 코드]

앞선 '2.분석 실습'에서 데이터 품질 전처리에 필요한 6가지 특성에 대하여 논의하였다. (완전성, 유일성, 유효성, 일관성, 정확성, 무결성)

• 데이터 품질 지수 : 세부 설명

▶ 품질 전처리를 위해 데이터를 호출한다.

```
import pandas as pd

normal = pd.read_csv('./press_data_normal.csv', index_col=0)
normal_data = normal.copy()
outlier = pd.read_csv('./outlier_data.csv', index_col=0)
outlier_data = outlier.copy()
```

[코드 1] 데이터 호출 코드

▶ 완전성 품질 지수 = ((1-결측치)/전체 데이터 수) * 100

: 각 칼럼에 누락 값이 없어야 한다.

```
ratio = 30

for column in normal_data.columns:
    tmp = normal_data[column]
    print('column name :', column)
    print('변수별 결측 비율')
    print(round(tmp.isnull().sum()/len(tmp)*100,2))
    print('변수별 결측 비율 30% 초과 여부')
    print(tmp.isnull().sum()/len(tmp)*100>ratio)
    print('='*30)
```

```
column name : TimeStamp
변수별 결측 비율
0.0
변수별 결측 비율 30% 초과 여부
False
=====
column name : AI0_Vibration
변수별 결측 비율
0.0
변수별 결측 비율 30% 초과 여부
False
=====
column name : AI1_Vibration
변수별 결측 비율
0.0
변수별 결측 비율 30% 초과 여부
False
=====
column name : AI2_Current
변수별 결측 비율
0.0
변수별 결측 비율 30% 초과 여부
False
=====
column name : Equipment_state
변수별 결측 비율
0.0
변수별 결측 비율 30% 초과 여부
False
=====
```

[코드 2] 정상데이터 변수 별 Null값 30% 이상 여부 확인 코드



```
ratio = 30

for column in outlier_data.columns:
    tmp = outlier_data[column]
    print('column name :', column)
    print('변수별 결측 비율')
    print(round(tmp.isnull().sum()/len(tmp)*100,2))
    print('변수별 결측 비율 30% 초과 여부')
    print(tmp.isnull().sum()/len(tmp)*100>ratio)
    print('='*30)
```

```
column name : TimeStamp
변수별 결측 비율
0.0
변수별 결측 비율 30% 초과 여부
False
=====
column name : AI0_Vibration
변수별 결측 비율
0.0
변수별 결측 비율 30% 초과 여부
False
=====
column name : AI1_Vibration
변수별 결측 비율
0.0
변수별 결측 비율 30% 초과 여부
False
=====
column name : AI2_Current
변수별 결측 비율
0.0
변수별 결측 비율 30% 초과 여부
False
=====
column name : Equipment_state
변수별 결측 비율
0.0
변수별 결측 비율 30% 초과 여부
False
=====
```

[코드 3] 이상데이터 변수 별 Null값 30% 이상 여부 확인 코드

- ① Null 값이 30% 이상인 데이터들은 데이터의 완전성이 떨어지기 때문에 변수별 Null 값의 비율을 확인하여 삭제한다. 본 데이터에서는 누락된 데이터가 존재하지 않아 데이터를 삭제하지 않는다.

```
normal_sum = 0

for column in normal_data.columns:
    tmp = normal_data[column]
    print("전체 데이터셋 결측치 개수")
    cmpt_len = tmp.isnull().sum().sum()
    print(cmpt_len)
    print(f"결측치 = {cmpt_len}개")
    print(f"완전성 지수 : {(1-cmpt_len/len(normal))*100}%")
    print(f"=*30)
    normal_sum += (1-cmpt_len/len(normal))*100
```

```
전체 데이터셋 결측치 개수
```

```
0
```

```
결측치 = 0개
```

```
완전성 지수 : 100.0%
```

```
=====
```

```
전체 데이터셋 결측치 개수
```

```
0
```

```
결측치 = 0개
```

```
완전성 지수 : 100.0%
```

```
=====
```

```
전체 데이터셋 결측치 개수
```

```
0
```

```
결측치 = 0개
```

```
완전성 지수 : 100.0%
```

```
=====
```

```
전체 데이터셋 결측치 개수
```

```
0
```

```
결측치 = 0개
```

```
완전성 지수 : 100.0%
```

```
=====
```

```
전체 데이터셋 결측치 개수
```

```
0
```

```
결측치 = 0개
```

```
완전성 지수 : 100.0%
```

```
=====
```

[코드 4] 정상데이터 변수 별 완전성 지수 산출 코드



```
outlier_sum = 0

for column in outlier_data.columns:
    tmp = outlier_data[column]
    print('전체 데이터셋 결측치 개수')
    cmpt_len = tmp.isnull().sum().sum()
    print(cmpt_len)
    print(f'결측치 = {cmpt_len}개')
    print(f'완전성 지수 : {(1-cmpt_len/len(outlier_data))*100}%')
    print('='*30)
    outlier_sum += (1-cmpt_len/len(outlier_data))*100
```

```
전체 데이터셋 결측치 개수
0
결측치 = 0개
완전성 지수 : 100.0%
=====
전체 데이터셋 결측치 개수
0
결측치 = 0개
완전성 지수 : 100.0%
=====
전체 데이터셋 결측치 개수
0
결측치 = 0개
완전성 지수 : 100.0%
=====
전체 데이터셋 결측치 개수
0
결측치 = 0개
완전성 지수 : 100.0%
=====
전체 데이터셋 결측치 개수
0
결측치 = 0개
완전성 지수 : 100.0%
=====
```

[코드 5] 이상데이터 변수 별 완전성 지수 산출 코드

② 정상데이터와 이상데이터 각각 완전성 품질 지수를 구한다.

```
print(f'정상데이터 완전성 지수 : {normal_sum/len(normal.columns)}%')
print(f'이상데이터 완전성 지수 : {outlier_sum/len(outlier_data.columns)}%')
```

```
정상데이터 완전성 지수 : 100.0%
이상데이터 완전성 지수 : 100.0%
```

[코드 6] 완전성 지수



▶ 유일성 품질 지수 = ((유일한 데이터 수)/전체 데이터 수) * 100

: 데이터 항목은 유일해야하며 중복되어서는 안된다.

```
check_unique = normal_data.value_counts().reset_index()
perc_check_unique=round((len(check_unique)-len(check_unique[check_unique[0]>1]))/len(check_unique)*100,2)
print(f"정상데이터 유일성 지수 :{perc_check_unique}%")

check_unique = outlier_data.value_counts().reset_index()
perc_check_unique=round((len(check_unique)-len(check_unique[check_unique[0]>1]))/len(check_unique)*100,2)
print(f"이상데이터 유일성 지수 :{perc_check_unique}%")
```

```
정상데이터 유일성 지수 : 99.99%
이상데이터 유일성 지수 : 100.0%
```

[코드 7] 유일성 품질 지수 산출 코드

① value_counts() 함수를 이용하여 key 값 개수를 확인하고 reset_index() 함수로 데이터 프레임 인덱스를 초기화한다.

② key 값을 정렬했을 때, 1 초과인 행 개수 비율의 평균이 유일성 품질 지수다.

▶ 유효성 품질 지수 = (유효성 만족 데이터 수/전체 데이터 수)*100

: 데이터 항목은 정해진 데이터 유효범위 및 도메인을 충족해야 한다.본 데이터에서는 수집 기간에 대한 유효성 품질지수를 확인한다.

```
import datetime
normal_data.TimeStamp = pd.to_datetime(normal_data.TimeStamp)
normal_start = pd.Timestamp(datetime.date(2022, 7, 12))
normal_end = pd.Timestamp(datetime.date(2022, 7, 13))
valid_normal = normal_data[(normal_data.TimeStamp >= normal_start) & (normal_data.TimeStamp < normal_end)]
valid_normal_len = len(valid_normal)
print('정상 데이터 유효성 지수 : {}'.format(valid_normal_len/len(normal_data) * 100))

outlier_data.TimeStamp = pd.to_datetime(outlier_data.TimeStamp)
outlier_start = pd.Timestamp(datetime.date(2022, 7, 17))
outlier_end = pd.Timestamp(datetime.date(2022, 7, 18))
valid_outlier = outlier_data[(outlier_data.TimeStamp >= outlier_start) & (outlier_data.TimeStamp < outlier_end)]
valid_outlier_len = len(valid_outlier)
print('이상 데이터 유효성 지수 : {}'.format(valid_outlier_len/len(outlier_data) * 100))
```

```
정상 데이터 유효성 지수 : 100.0%
이상 데이터 유효성 지수 : 100.0%
```

[코드 8] 데이터 수집 범위 확인 및 유효성 지수 확인 코드

① 수집된 날짜 안에 들어가 있는가?

- 정상 데이터의 경우 2022년 07월 12일, 이상데이터의 경우 2022년 07월 17일에 수집된 데이터이기 때문에, 데이터가 해당 기간을 벗어나지 않았는지 확인한다.



▶ 일관성 품질 지수 = (일관성 만족 데이터 수/전체 데이터 수)*100

: 데이터가 지켜야 할 구조, 값, 표현되는 형태가 일관되게 정의되고 서로 일치해야 한다.

```
normal_ = normal_data.copy()
for column in normal.columns:
    if normal_[column].dtype == 'int64':
        normal_[column] = normal_[column].apply(lambda x: isinstance(x, int))

    if normal_[column].dtype == 'float64':
        normal_[column] = normal_[column].apply(lambda x: isinstance(x, float))

normal_valid = len(normal_) / len(normal_) * 100
print(f"정상 데이터 일관성 지수 : {normal_valid}%")

outlier_ = outlier_data.copy()
for column in outlier_data.columns:
    if outlier_[column].dtype == 'int64':
        outlier_[column] = outlier_[column].apply(lambda x: isinstance(x, int))

    if outlier_[column].dtype == 'float64':
        outlier_[column] = outlier_[column].apply(lambda x: isinstance(x, float))

outlier_valid = len(outlier_) / len(outlier_) * 100
print(f"이상 데이터 일관성 지수 : {outlier_valid}%")
```

```
정상 데이터 일관성 지수 : 100.0%
이상 데이터 일관성 지수 : 100.0%
```

[코드 9] 칼럼별 데이터 형식 확인 및 일관성 지수 확인 코드

① 데이터가 형식에 맞는가?

- 데이터 열은 int 혹은 float 타입의 값이어야 한다.
- 'isinstance()' 함수를 사용하여 위 데이터 형식을 만족하는지 확인한다.

▶ 정확성 품질 지수 = (1-(정확성 위배 데이터 수/전체 데이터 수))*100

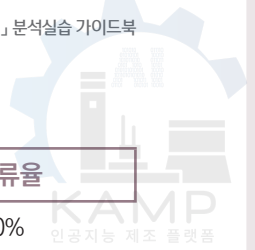
: 실제 존재하는 객체의 표현 값이 정확히 반영되어야 한다. 본 데이터에서 칼럼값은 독립적 이므로 정확성 지수를 확인하지 않는다.

▶ 무결성 품질지수 = (1-(유일성, 유효성, 일관성 지수중 100%가 아닌 지수 개수 / 3))*100

: 정상 데이터는 세가지 지수 중 유일성 99.9%, 유효성 100%, 일관성 100% 이므로 무결성 품질 지수는 66.67%이고, 이상 데이터는 모든 지수가 100%이므로 무결성 지수 또한 100% 이다.

▶ 가중치 지수 = 품질 지수 * 가중치

▶ 데이터 품질 지수 = ∑ 가중치 지수

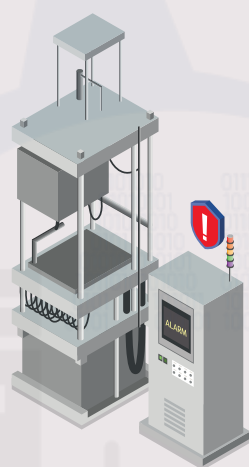


구분	품질지수	가중치	가중치 지수	오류율
완정성 (누락)	100%	60%	60%	0%
유일성 (중복)	99.99%	10%	9.99%	0.01%
유효성 (유효)	100%	10%	10%	0%
일관성 (표현)	100%	10%	10%	0%
정확성 (실제)	-	-	-	-
무결성	66.67%	10%	6.67%	3.33%
품질지수		100%	96.66%	3.34%

[표 1] 정상 데이터 품질지수

구분	품질지수	가중치	가중치 지수	오류율
완정성 (누락)	100%	60%	60%	0%
유일성 (중복)	100%	10%	10%	0%
유효성 (유효)	100%	10%	10%	0%
일관성 (표현)	100%	10%	10%	0%
정확성 (실제)	-	-	-	-
무결성	100%	10%	10%	0%
품질지수		100%	100%	0%

[표 2] 이상 데이터 품질지수



「소성가공 예지보전 AI 데이터셋」 분석실습 가이드북